

## Tutorial

Time: Sunday, 16 November 2025, 1:30pm - 5:00pm CST

Location: 124

**IMPORTANT:** For accessing NERSC hardware, please fill out the [linked google form](#) ASAP.  
Needs to be filled out by Nov 1 for guaranteed access.

# Accelerated Quantum Supercomputing in Action: A Hands-On Tutorial on Scalable Hybrid Workflows

Monica Van Dieren, NVIDIA; Pranav Gokhale, Infleqtion; Ermal Rrapaj, NERSC; Pooja Rao, NVIDIA



# Outline

- Accelerated Quantum Supercomputing
  - Onboarding with NERSC
  - Introduction to AQSC
  - Introduction to CUDA-Q
- AI for Scalable Quantum Algorithms
  - Hybrid workflows
  - GQE hands-on
  - Multi-gpu parallelization
- Logical Qubit Experiments
  - Logical qubit encoded VQE demo
  - Many-hypercube code logical qubits decoding demo on NERSC supercomputer
  - Atom rearrangement and readout on Infleqtion's Sqaile QPU

# Compute Resources

- NERSC
  - Perlmutter  
Fill out the following form  
<https://docs.google.com/forms/d/e/1FAIpQLSe2gcOlj1djoLzzw6Y35dIyN6G2JEjPKgVWo9WUnbXw4UsQLA/viewform>
- Inflection
  - Neutral atom SqaLe QPU for real-time demo of atom rearrangement and readout
- NVIDIA
  - CUDA-Q on Launchables

# Github Repository

## Main repo

- Official repository for all the material in the tutorial (to be made public prior to the tutorial)

<https://github.com/NERSC/SC25-quantum-tutorial>



## Additional resources

- CUDA-Q resources
  - Github repo and docs
  - CUDA-QX repo and docs
  - CUDA-Q Academic
- GQE
  - Paper  
<https://arxiv.org/abs/2401.09253>
  - Code  
[https://github.com/NVIDIA/cudaqx/blob/main/docs/sphinx/examples/solvers/python/gqe\\_h2.py](https://github.com/NVIDIA/cudaqx/blob/main/docs/sphinx/examples/solvers/python/gqe_h2.py)

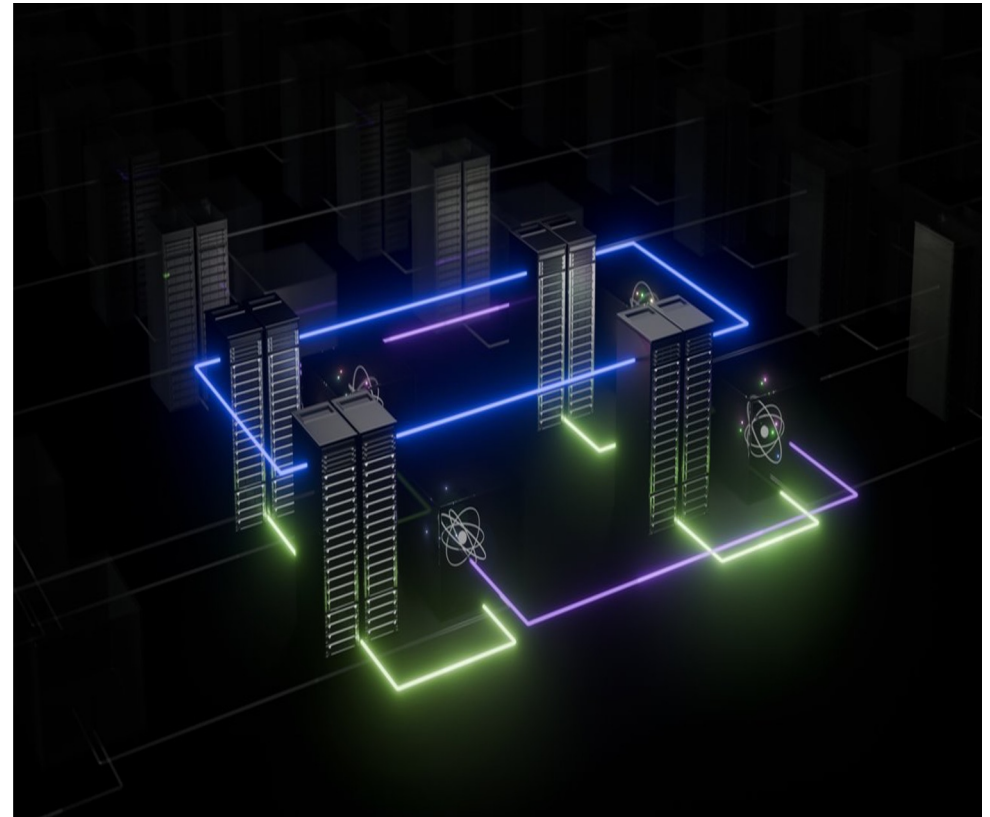
**Hour 1**

# **Accelerated Quantum Supercomputing**



## The Accelerated Quantum Supercomputer

- Supercomputing architecture connecting quantum hardware
- Ability to run hybrid algorithms - using GPUs and QPUs
- A software platform that seamlessly connects hybrid applications
- The ability to perform qubit-agnostic development of control and error correction

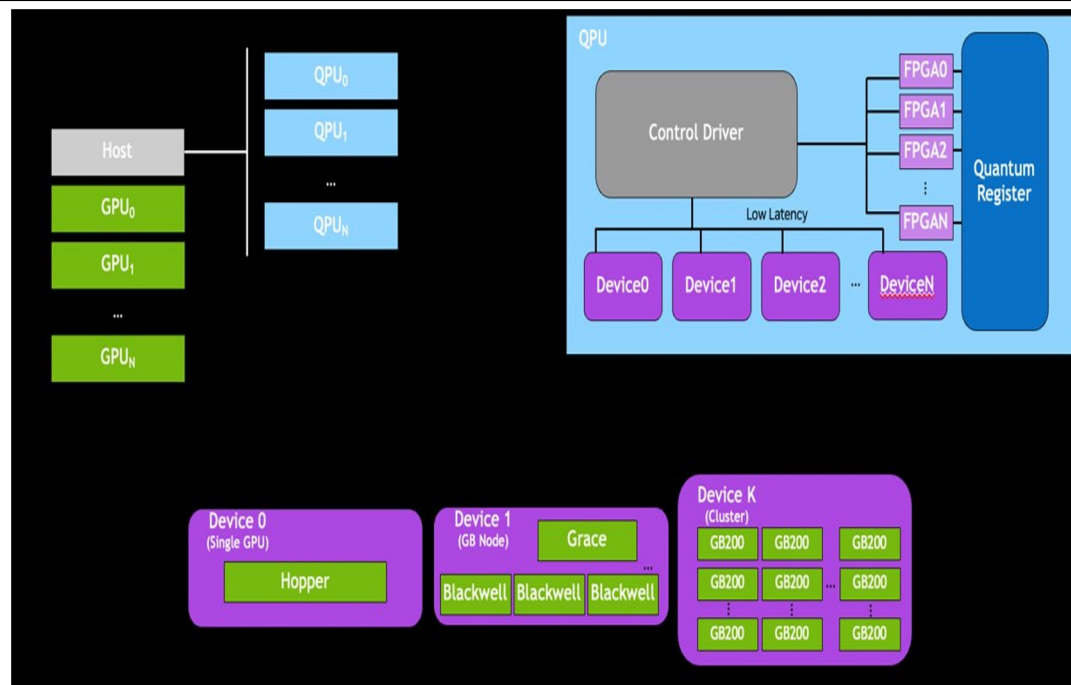


# Defining the Accelerated Quantum Supercomputers

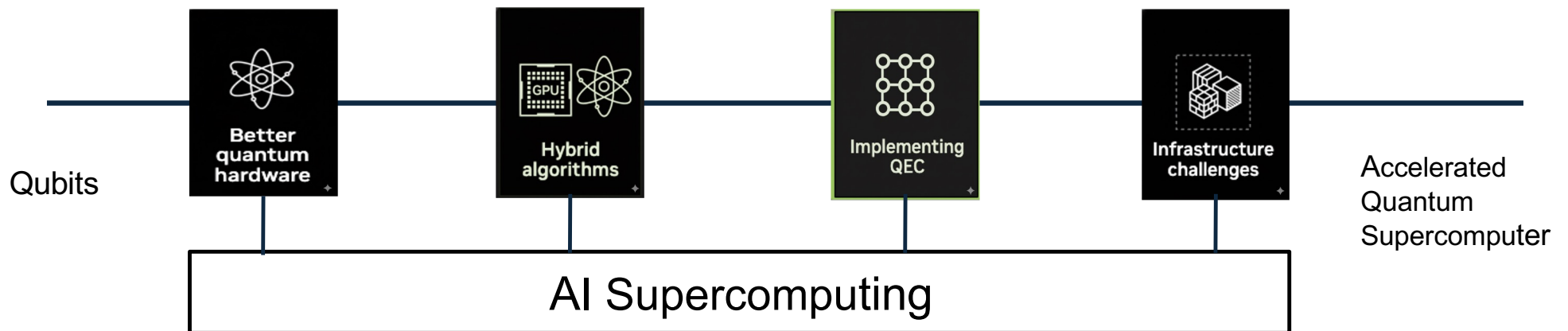
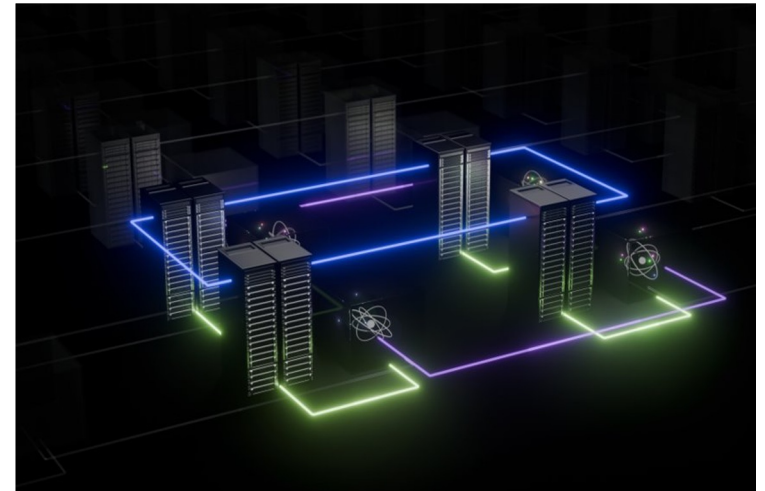
- Programming model and compiler for heterogeneous supercomputer
- Low-latency interconnects for real-time hybrid computing
- Libraries to enable domain scientists
- Open source and qubit-agnostic
- Scalable and efficient GPU-accelerated simulators (digital and analog)

Libraries, Applications, Frameworks

CUDA-Q : Platform for Programming Accelerated Quantum Supercomputers

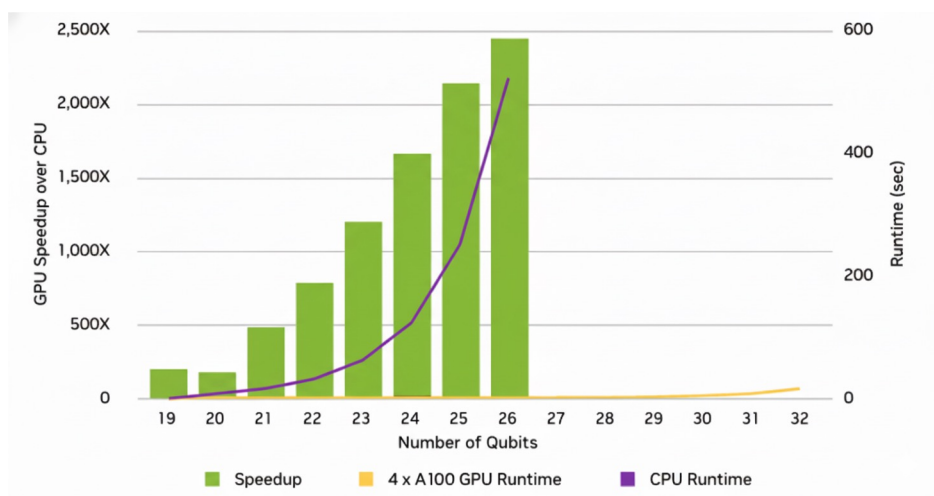


# Accelerating the Journey From Qubits to Supercomputers



# CUDA-Q : Platform for Accelerated Quantum Supercomputing

## Performance



QML workflow in CUDA-Q using multithreaded CPU versus NVIDIA A100 GPUs

- **Python and C++**
- **QPU agnostic**
  - Supports backends from all major QPU vendors and qubit types
- **GPU-accelerated simulation**
  - Quantum simulators that scale to large-scale quantum computers
- Fully kernel system for hybrid computing interface
  - Seamlessly combine GPU and QPU resources
- Access to classical **CUDA-X libraries**
  - Conventional parts of hybrid algorithms can draw on fastest implementations
- Comprehensive **educational tools**
  - CUDA-Q Academic onboards users to accelerated quantum supercomputing

# Outline for NERSC Section of the CUDA-Q Tutorial

- Description of NERSC systems
- How to access NERSC training accounts
- Tutorial code and examples:  
<https://github.com/NERSC/sc25-quantum-tutorial>

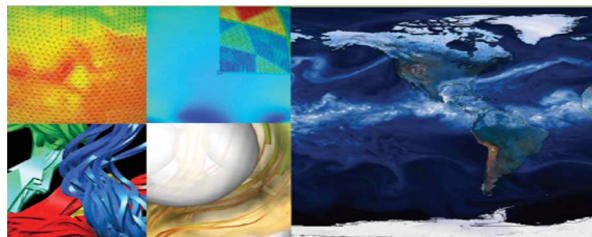
# NERSC is the Mission HPC Facility for DOE Office of Science Research



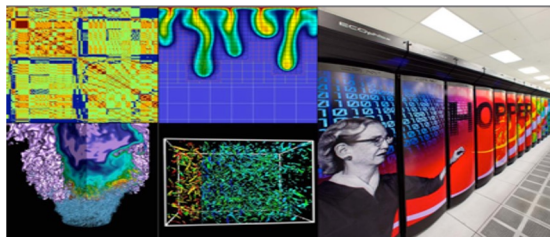
U.S. DEPARTMENT OF  
**ENERGY**

Office of  
Science

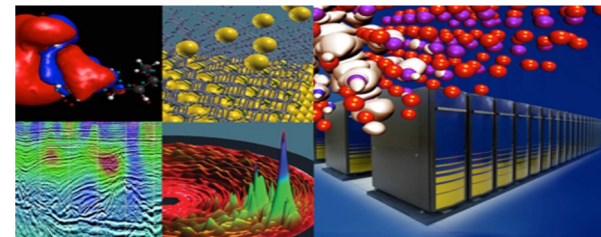
Largest funder of physical science  
research in the U.S.



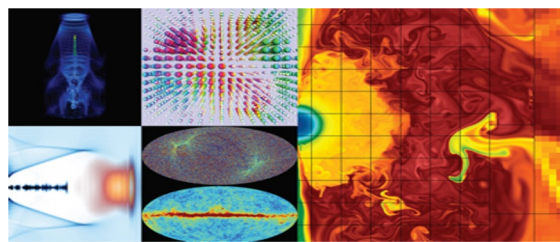
Biological and Environmental Research



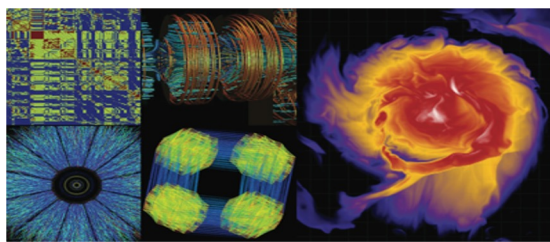
Computing



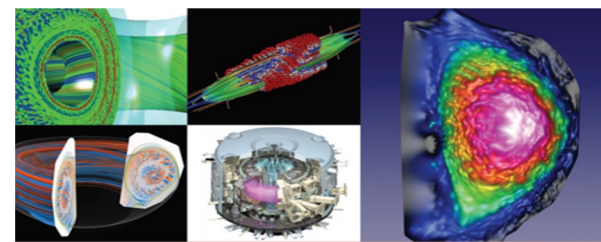
Basic Energy Sciences



High Energy Physics

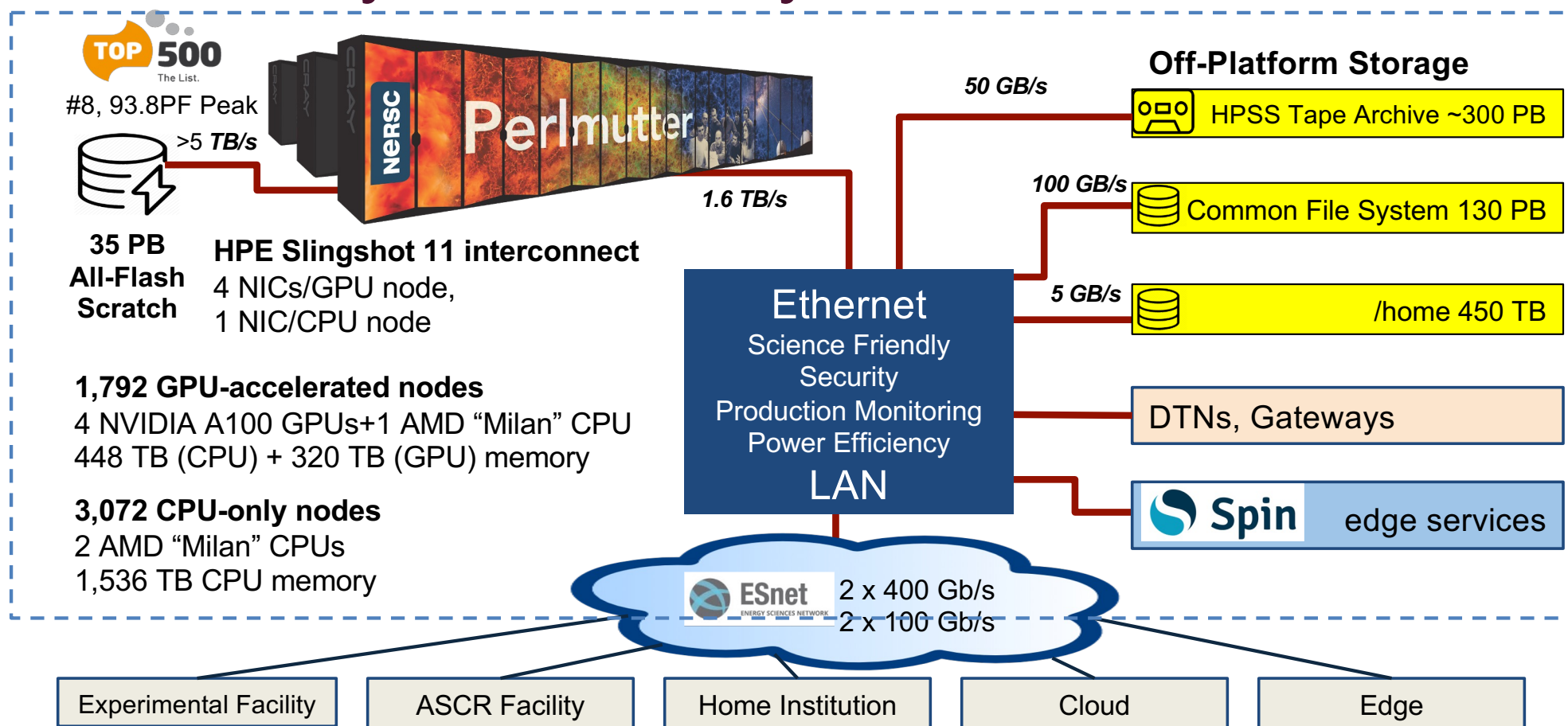


Nuclear Physics

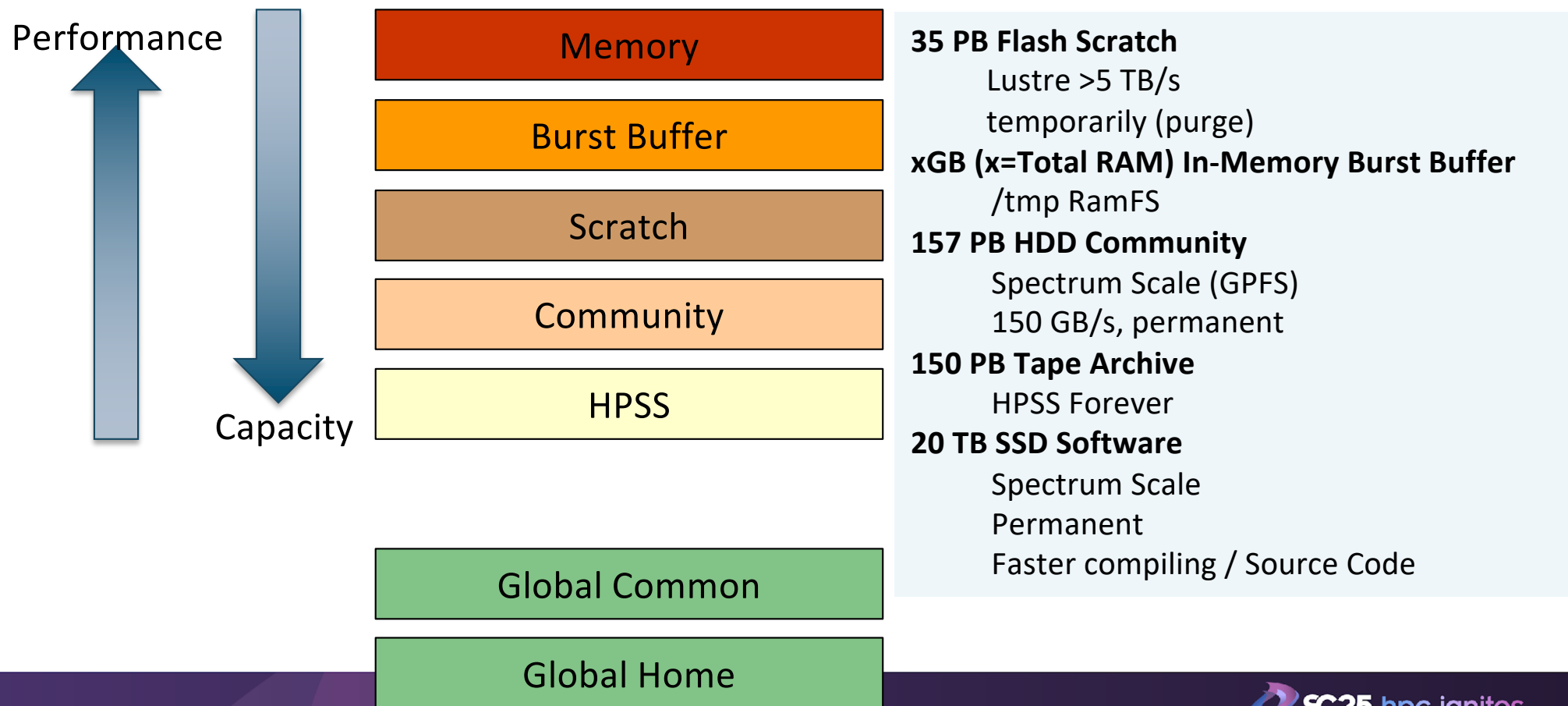


Fusion Energy, Plasma Physics

# NERSC Systems Ecosystem

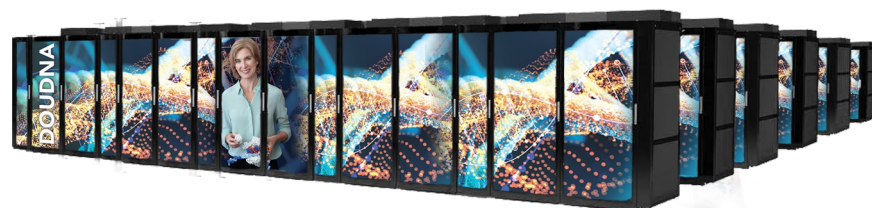


# Simplified NERSC File Systems

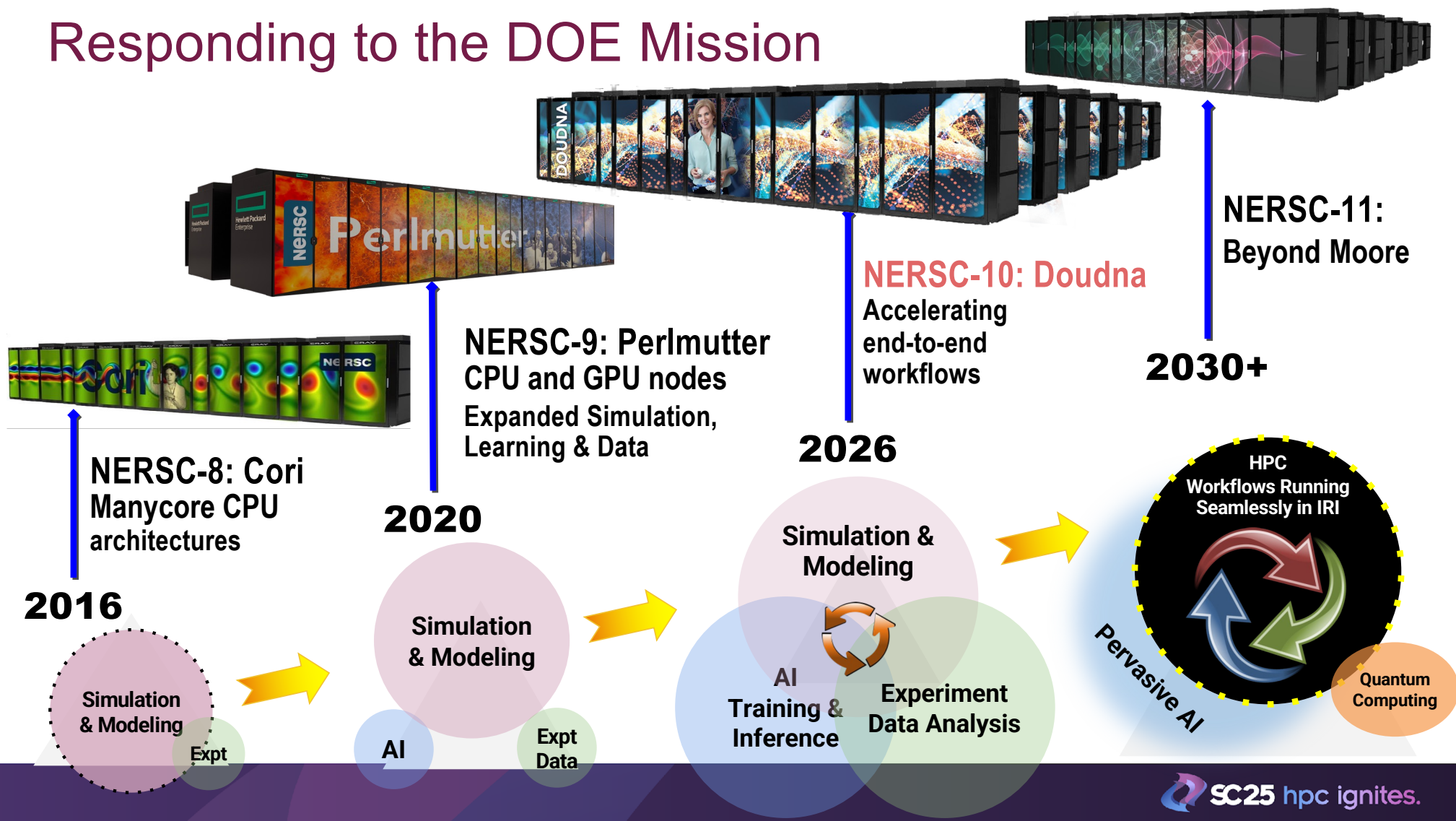


# NERSC has a dual mission to advance science and the state-of-the-art in supercomputing

- We collaborate with computer companies years before a system's delivery to deploy advanced systems with new capabilities at large scale
- We provide a highly customized software and programming environment for science applications
- We are tightly coupled with the workflows of DOE's experimental and observational facilities – ingesting tens of terabytes of data each day
- Our staff provide advanced application and system performance expertise to users (> 12000 user, > 1000 projects )



# Responding to the DOE Mission



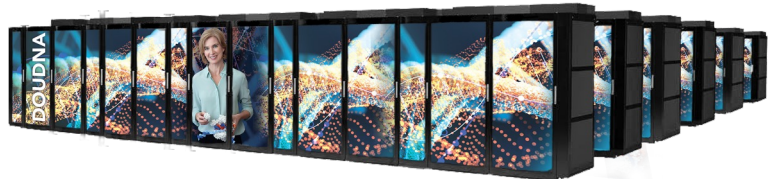
# NERSC-10: A Supercomputer for Complex, Integrated Workflows

*Users require an **integrated ecosystem** that supports new paradigms for **data analysis** with **real-time interactive feedback** between experiments and simulations. Users need the ability to **search, analyze, reuse, and combine data** from **different sources** into **large scale simulations and AI models**.*

**NERSC-10 Mission Need Statement:** The NERSC-10 system will accelerate end-to-end DOE SC workflows and enable new modes of scientific discovery through the integration of experiment, data analysis, and simulation.



# Doudna System Overview



Designed to provide > 10x performance over *Perlmutter* and support diverse and complex DOE SC workflows.

## NVIDIA User Software

### Vera-Rubin CPU-GPU

#### **NVIDIA Vera-Rubin Compute Node**

- Able to support GPU and CPU portions of complex workflows
- Integrated AI and Compute Optimized Partitions

### High Speed Network

#### **NVIDIA Infiniband XDR (800 Gbps)**

- Quantum-3 Infiniband switches
- ConnectX-8 Infiniband NICs
- Unified Fabric Manager

#### **External Connectivity**

- Skyway-Next IB-to-Eth Gateway

### 2 Types of Storage



**Platform Storage System (PSS)**  
IBM Storage Scale System



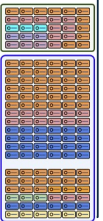
**Quality of Service Storage System (QSS)**  
on Dell PowerEdge Servers

### Workflow Environment

#### **Workflow Environment Nodes**

Heterogeneous node-types  
CPU-only, Vera-Rubin  
Air-cooled, Water-cooled

Reconfigurable to support complex IRI workflows (e.g. batch, compile, Jupyter, cloud-native, data transfer, etc.)

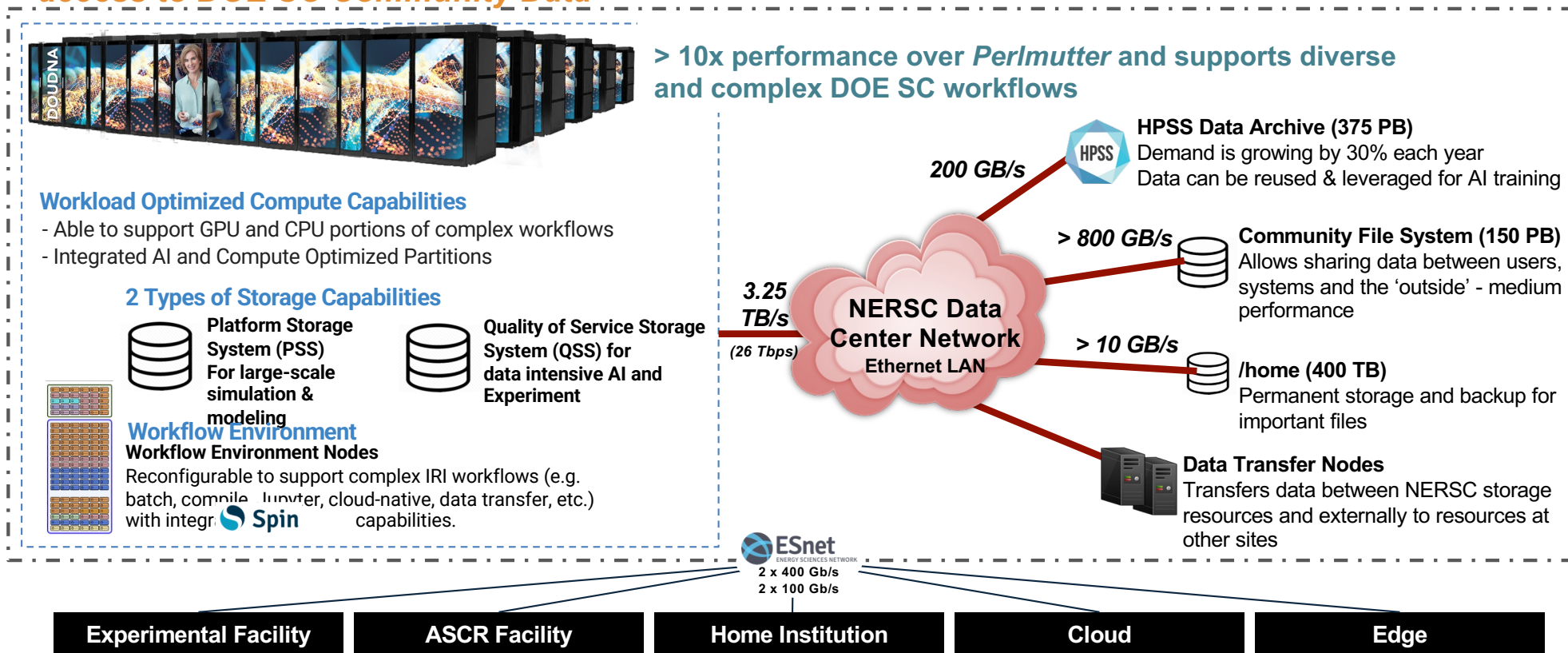


Dell System Management Tools (OME, iDRAC, Omnia)

Dell ORv3 Direct liquid-cooled server technology & integrated rack scalable systems

# Doudna Drives the Integrated Data Center Ecosystem

Scientific breakthroughs in AI and at Experimental Facilities enabled through high-speed access to DOE SC Community Data



# Quantum@NERSC Pathfinding and Early User & Vendor Exploration Activities



## Pathfinding R&D



Application Development



Benchmarking

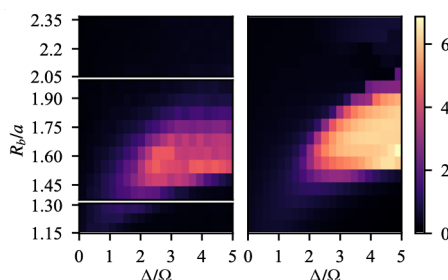


Resource Evaluation



## Vendor Collaboration

### |QuEra> R&D Partnership



## User Programs

### [Quantum Computing Access @ NERSC \(QCAN\)](#)

- 2022 QIS @ Perlmutter



Quantum Information Science @



30+ Teams and Publications

- 2024: |QuEra> - 8 teams
- 2025:   - 15 teams

## Simulation of Quantum Systems



**NVIDIA** Collaboration:  
Developing with CUDA-Q  
quantum software  
development toolkit on  
Perlmutter



## Xanadu

NESAP Collaboration:  
Quantum machine learning  
workflows with PennyLane on  
Perlmutter

# Quantum Computing Access @ NERSC: Neutral Atoms

- Call released in July
- Up to 25h of QC usage per winner
- Close collaboration between NERSC and science teams
- Users can take advantage of CUDA-Q with QuEra's Aquila quantum processor

The screenshot shows the NERSC website header with the logo and tagline "Powering Scientific Discovery for 50 Years". The navigation bar includes links for HOME, ABOUT, SCIENCE, SYSTEMS, FOR USERS, NEWS, R & D (highlighted), EVENTS, and LIVE STATUS. A search bar is in the top right corner. The left sidebar lists various research areas, with "Quantum Computing Access @ NERSC: Neutral Atoms - Call for User Proposals" highlighted in green. The main content area features the title "QUANTUM COMPUTING ACCESS @ NERSC: NEUTRAL ATOMS - CALL FOR USER PROPOSALS" in large green letters. Below the title, a paragraph states: "The National Energy Research Scientific Computing Center (NERSC) at Berkeley Lab is seeking user proposals as part of its Quantum Computing Access @ NERSC (QCAN) program to conduct research on one of the first publicly accessible quantum computers based on neutral atom technology through its partnership with QuEra Computing. Neutral atom quantum computers offer distinct advantages in terms of scale and coherence time. This first-of-its-kind user program offers a unique opportunity to gain access to [Aquila](#), the neutral atom analog quantum computer developed by QuEra Computing."

## Three ways to access NERSC

To use our resources, you need to either:

1. Log into the login nodes and interact with Slurm (covered here)
1. Use services (eg. Jupyter) that expose web interfaces (covered later)
1. Interact via our REST API – called the “Superfacility API”  
(see: <https://docs.nersc.gov/services/sfapi/>)

# Connecting with SSH

- To access Perlmutter, use:  
saul.nersc.gov  
perlmutter.nersc.gov



```
blaschke@laptop:~$ ssh <user>@saul.nersc.gov  
blaschke@laptop:~$ ssh <user>@perlmutter.nersc.gov
```

- To be able to open GUI applications on the login nodes use: `ssh -Y`



```
blaschke@laptop:~$ ssh -Y <user>@saul.nersc.gov
```

# Connecting with SSH

After successfully logging on, you be greeted by the terms of use, and the command-line input prompt:

```
For past outages, see: https://my.nersc.gov/outagelog-cs.p  
blaschke@perlmutter:login17:~
```

From here you can interact with perlmutter...

[illegible]

# Submitting Jobs

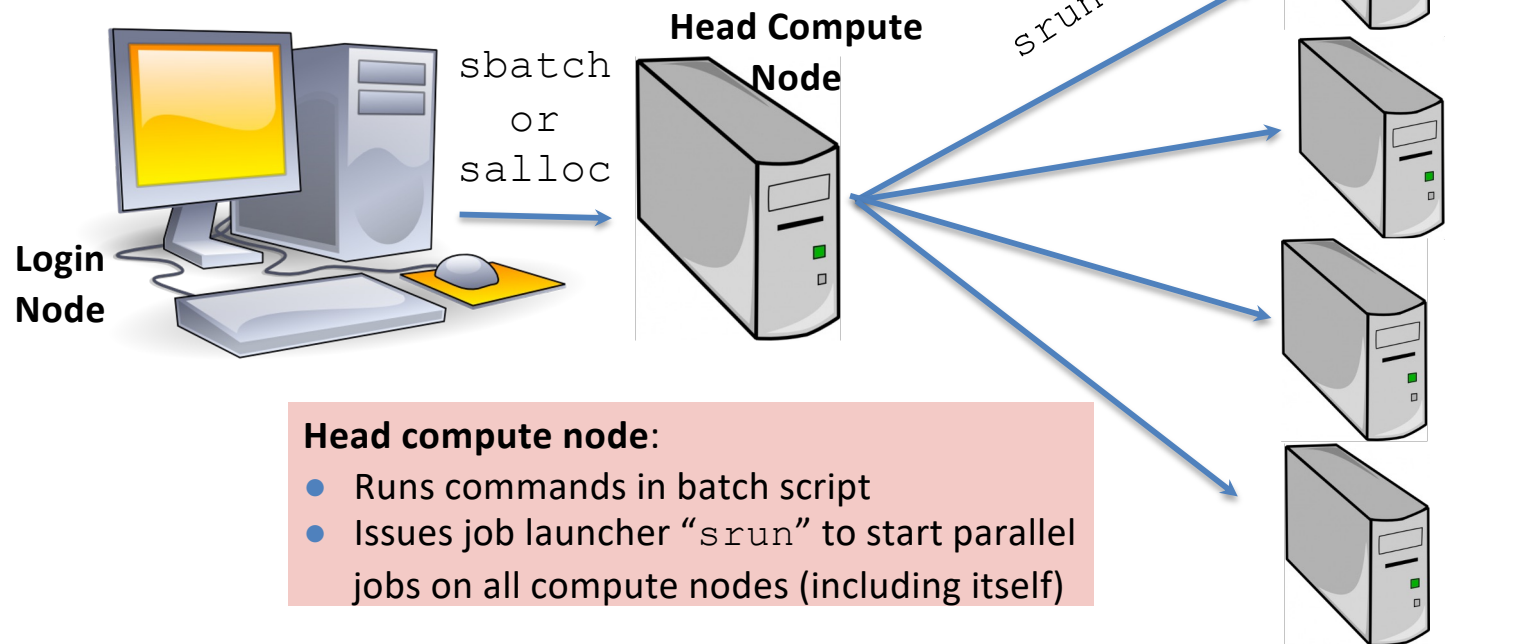
- Jobs can be submitted to queueing system through sbatch or salloc:
  - `sbatch --reservation=SC25_Quantum <my_job_script>`
  - `salloc --reservation=SC25_Quantum <options>`
- The above methods list details about resources needed for a job and for how long
- Eg.: a request for a cpu node to be allocated for 5 mins:
  - `salloc --reservation=SC25_Quantum -N 1 -C cpu -q debug -t 5 -A <project>`



# Submitting Jobs

## Login node:

- Submit batch jobs via `sbatch` or `salloc`



## Head compute node:

- Runs commands in batch script
- Issues job launcher "`srun`" to start parallel jobs on all compute nodes (including itself)

\*figure courtesy Helen (2020 NERSC Training)

# My First “Hello World” Job Script

debug queue

2 nodes

10 min “walltime”

run on haswell partition

use SCRATCH file system

run 64 processes in parallel

my\_batch\_script.sh

```
#!/bin/bash
#SBATCH -q debug
#SBATCH -N 2
#SBATCH -t 10:00
#SBATCH -C haswell
#SBATCH -L SCRATCH
#SBATCH -J myjob
srun -n 64 ./helloWorld
```

## To run via batch queue

```
% sbatch my_batch_script.sh
```

## To run via interactive batch

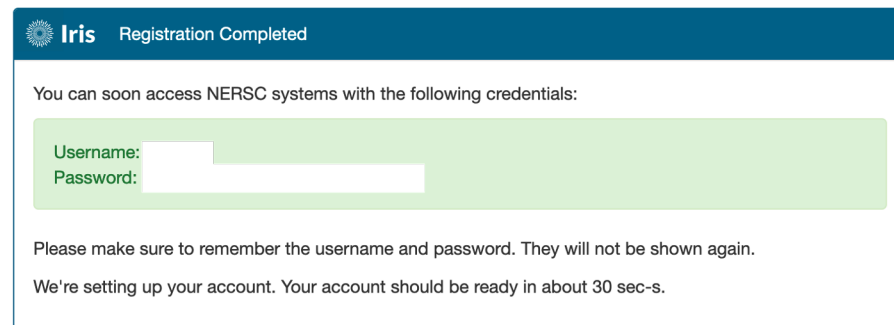
```
% salloc -N 2 -q interactive -C gpu -t 10:00
```

<wait\_for\_session\_prompt. Land on a compute node>

```
% srun -n 64 ./helloWorld
```

## Activate your Training Account

- Go to: <https://iris.nersc.gov/train>
- Enter the Training Code: “*ep81*” along with your details  
**Note:** if you have previously registered an account using a different code please recreate it.
- **Important: make a copy of your login and password** you won't be able to change these, nor recover them later!!!



The screenshot shows a web interface titled "Iris Registration Completed". It informs the user that they can soon access NERSC systems with the following credentials. The credentials are displayed in a green box with "Username:" and "Password:" labels, followed by redacted text. Below this, a message states: "Please make sure to remember the username and password. They will not be shown again. We're setting up your account. Your account should be ready in about 30 sec-s."

**Iris** Registration Completed

You can soon access NERSC systems with the following credentials:

Username: [REDACTED]  
Password: [REDACTED]

Please make sure to remember the username and password. They will not be shown again.  
We're setting up your account. Your account should be ready in about 30 sec-s.

# Multi-Factor Authentication

- NERSC users are required to use Multi-Factor Authentication (MFA) for logging into NERSC resources. MFA provides greater protection than regular passwords against phishing and other modern threats to your digital security. With NERSC's MFA, you authenticate using your NERSC password plus a "one-time password" (OTP).
- MFA at NERSC makes use of an app that you install on your mobile device, which you configure through Iris. The authenticator presents a 6-digit password that changes every 30 seconds. Each password can be used only one time, thus the name "one time password" or "OTP." When you log in to an MFA-enabled system, you will enter your NERSC password followed by the OTP displayed on your authenticator app.
- NERSC primarily supports the Google Authenticator app, which runs on an Android or iOS device. Other TOTP (Time-based OTP) authenticator apps can work, too - you can search for "TOTP" to find other options.
- To set up MFA, follow the instructions at: <https://docs.nersc.gov/connect/mfa/>

# Access to Perlmutter and Use CUDA-Q

- NERSC users have been added to `ntrain6` project
- Login to Perlmutter: ssh [username@perlmutter.nersc.gov](mailto:username@perlmutter.nersc.gov)
- Cuda modules:
  - `% module load cudatoolkit`
- Running Jobs examples:
  - <https://docs.nersc.gov/jobs/>

# Compute Node Reservations

- GPU node reservation:
  - To use 1 GPU only (sample flags for sbatch or salloc):
    - `-A ntrain6 --reservation=sc24_qc_tutorial -C gpu -N 1 -c 32 -G 1 -t 30:00 -q shared`
  - To use multiple nodes (sample flags for sbatch or salloc):
    - `-A ntrain6 --reservation=sc24_qc_tutorial -C gpu -N 2 -t 30:00 -q regular`
- Outside of reservation, use:
  - To use 1 GPU only (sample flags for sbatch or salloc):
    - `-A <project> -C gpu -N 1 -c 32 -G 1 -t 30:00 -q shared`
  - To use multiple nodes (sample flags for sbatch or salloc):
    - `-A <project> -C gpu -N 2 -t 30:00 -q regular (or -q interactive for salloc)`

# Reminder about file Systems

`$HOME` is not the best file system for running jobs at scale.  
Consider therefore replacing all occurrences of `$HOME` with `$SCRATCH`.

<https://github.com/NERSC/sc25-quantum-tutorial>

More detailed information in the github repository

# Logging into Jupyter

- Go to <https://jupyter.nersc.gov>
- Sign in using your training account credentials
- Select your preferred jupyter instance:

 Home Token Services Documentation train130 Logout

The following NERSC resources that Jupyter depends on appear to be in maintenance or having issues. This may impact Jupyter. See the [NERSC MOTD](#) for further information.  
Perlmutter status: degraded

|            | Shared CPU Node                                                                            | Exclusive CPU Node                                                                                               | Exclusive GPU Node    | Configurable GPU                                                                    |
|------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------|-------------------------------------------------------------------------------------|
| Perlmutter | <a href="#">start</a>                                                                      | <a href="#">start</a>                                                                                            | <a href="#">start</a> | <a href="#">start</a>                                                               |
| Cori       | <a href="#">start</a>                                                                      |                                                                                                                  |                       | <a href="#">start</a>                                                               |
| Resources  | Use a node shared with other users' notebooks but outside the batch queues.                | Use your own node within a job allocation using defaults.                                                        |                       | Use multiple compute nodes with specialized settings.                               |
| Use Cases  | Visualization and analytics that are not memory intensive and can run on just a few cores. | Visualization, analytics, machine learning that is compute or memory intensive but can be done on a single node. |                       | Multi-node analytics jobs, jobs in reservations, custom project charging, and more. |

# Setup

- Go to <https://jupyter.nersc.gov> and select: “Exclusive GPU Node” in the “Perlmutter” row

 Home Token Services Documentation train130 Logout

The following NERSC resources that Jupyter depends on appear to be in maintenance or having issues. This may impact Jupyter. See the [NERSC MOTD](#) for further information.  
Perlmutter status: degraded

|            | Shared CPU Node                                                                            | Exclusive CPU Node                                                                                               | Exclusive GPU Node    | Configurable GPU                                                                    |
|------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------|-------------------------------------------------------------------------------------|
| Perlmutter | <a href="#">start</a>                                                                      | <a href="#">start</a>                                                                                            | <a href="#">start</a> | <a href="#">start</a>                                                               |
| Cori       | <a href="#">start</a>                                                                      |                                                                                                                  |                       | <a href="#">start</a>                                                               |
| Resources  | Use a node shared with other users' notebooks but outside the batch queues.                | Use your own node within a job allocation using defaults.                                                        |                       | Use multiple compute nodes with specialized settings.                               |
| Use Cases  | Visualization and analytics that are not memory intensive and can run on just a few cores. | Visualization, analytics, machine learning that is compute or memory intensive but can be done on a single node. |                       | Multi-node analytics jobs, jobs in reservations, custom project charging, and more. |

# Setup

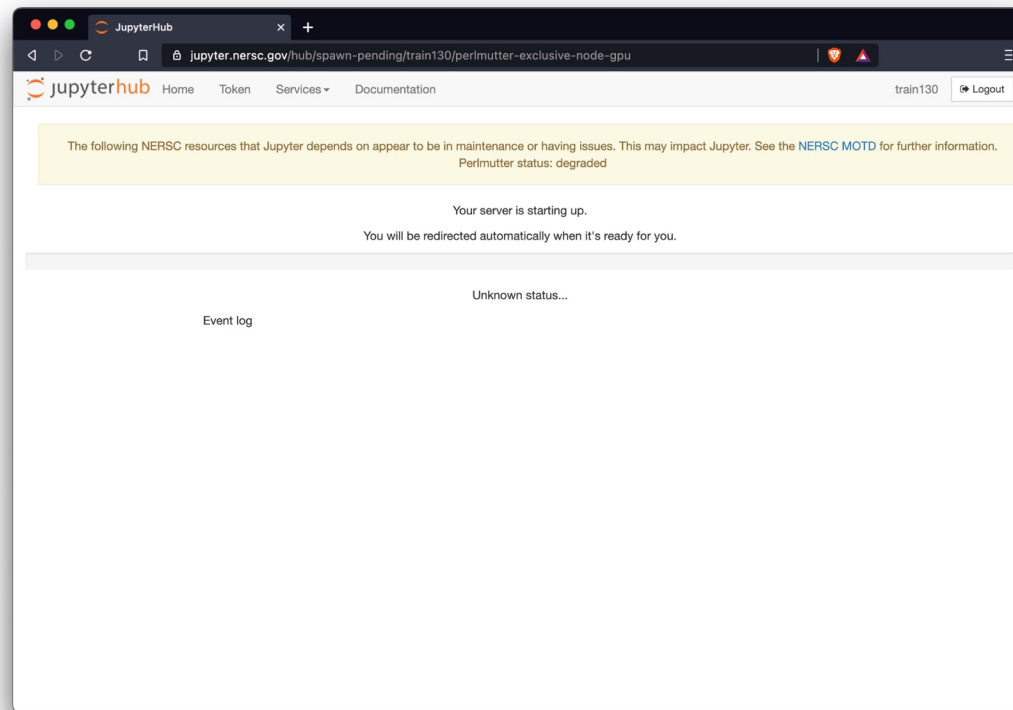
If you're spending a long time in the queue waiting for an exclusive GPU node, "Shared GPU Node"s are also available – eg:

|            | Login Node                                                                                 | Shared GPU Node                                                                                       | Exclusive CPU Node                                                                                               | Exclusive GPU Node                                                                  | Configurable Job       |
|------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|------------------------|
| Alvarez    | <button>start</button>                                                                     | <button>start</button>                                                                                | <button>start</button>                                                                                           | <button>start</button>                                                              |                        |
| Muller     | <button>start</button>                                                                     | <button>start</button>                                                                                | <button>start</button>                                                                                           | <button>start</button>                                                              | <button>start</button> |
| Perlmutter | <button>stop</button> <button>server</button>                                              | <button>start</button>                                                                                | <button>start</button>                                                                                           | <button>start</button>                                                              | <button>start</button> |
| Resources  | Use a login node shared with other users, outside the batch queues.                        | Use a single GPU on a node within a job allocation using defaults.                                    | Use your own node within a job allocation using defaults.                                                        | Use multiple compute nodes with specialized settings.                               |                        |
| Use Cases  | Visualization and analytics that are not memory intensive and can run on just a few cores. | Work that fits on a single GPU, and uses at most a quarter of a GPU node's CPU cores and host memory. | Visualization, analytics, machine learning that is compute or memory intensive but can be done on a single node. | Multi-node analytics jobs, jobs in reservations, custom project charging, and more. |                        |

Alternatively please use a reservation (details later)

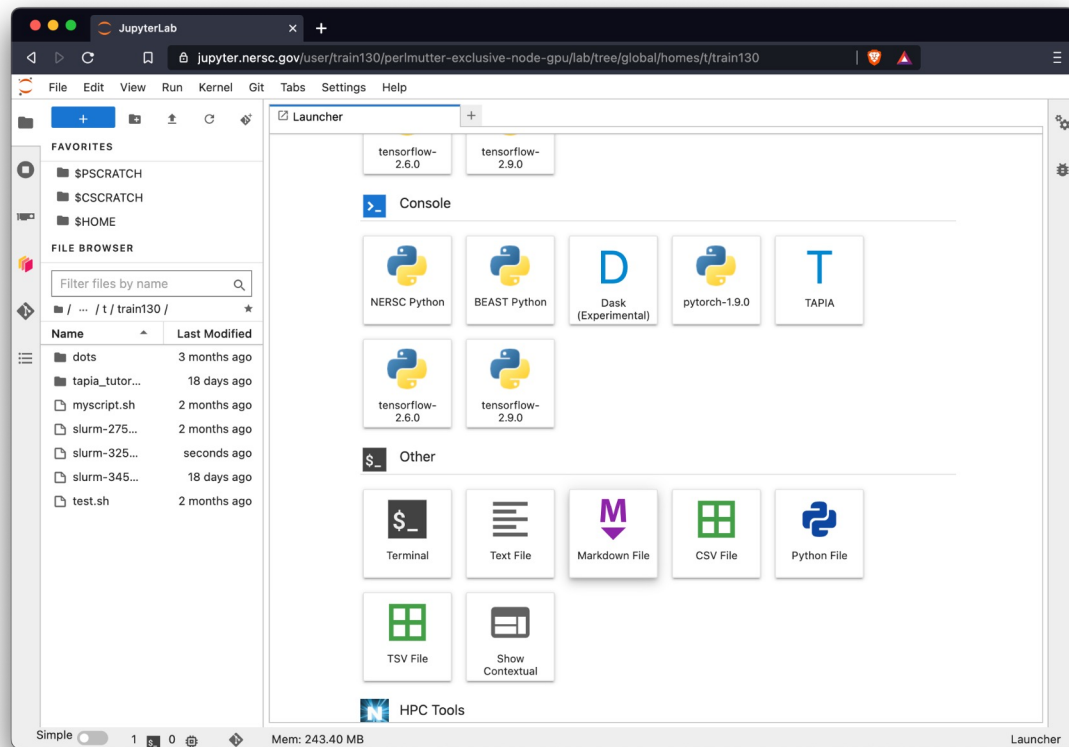
# Setup

- Jupyter should take a minute to start:



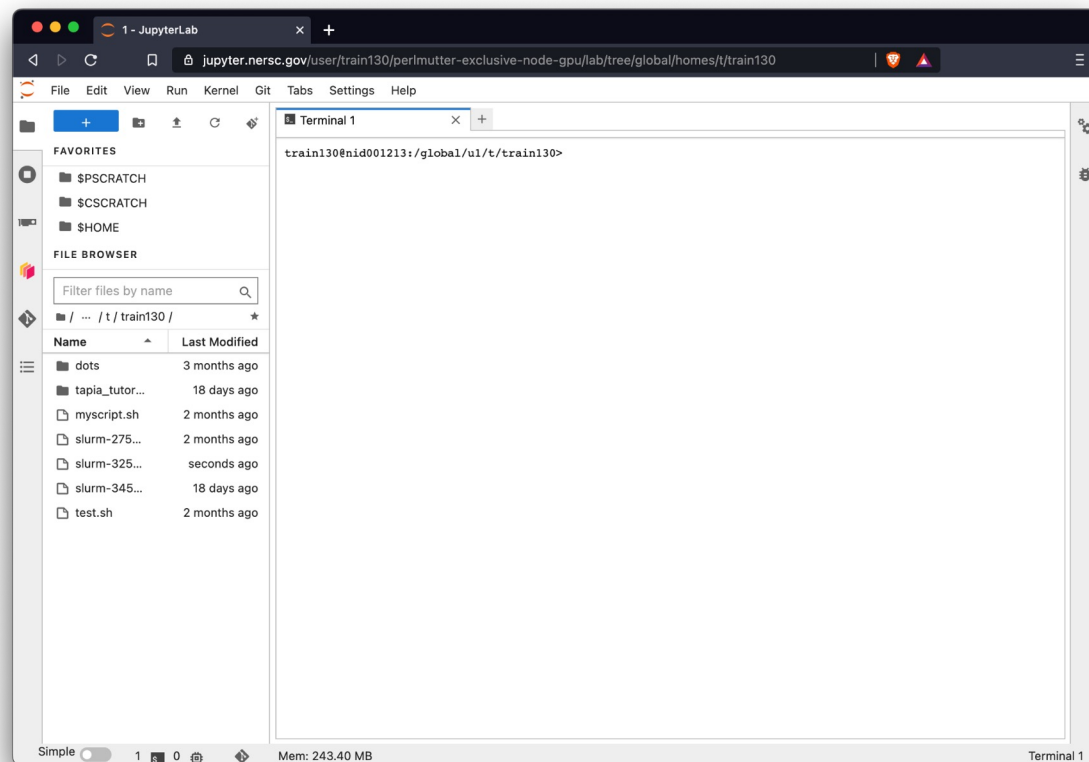
# Setup

- (If you don't see a terminal), select “+” followed by “Terminal”



# Setup

- Once started you should see a terminal

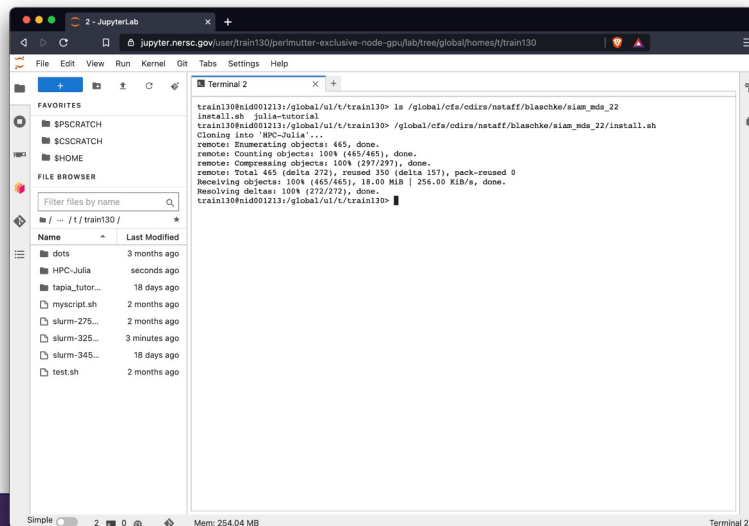


# Setup

- (In the terminal). follow installation instructions

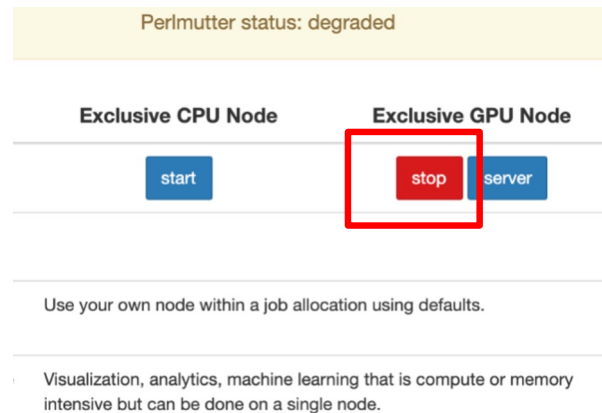
Upload the folders and script to your scratch directory:

```
$ scp folder_name username@perlmutter.nersc.gov:/pscratch/your_path
```



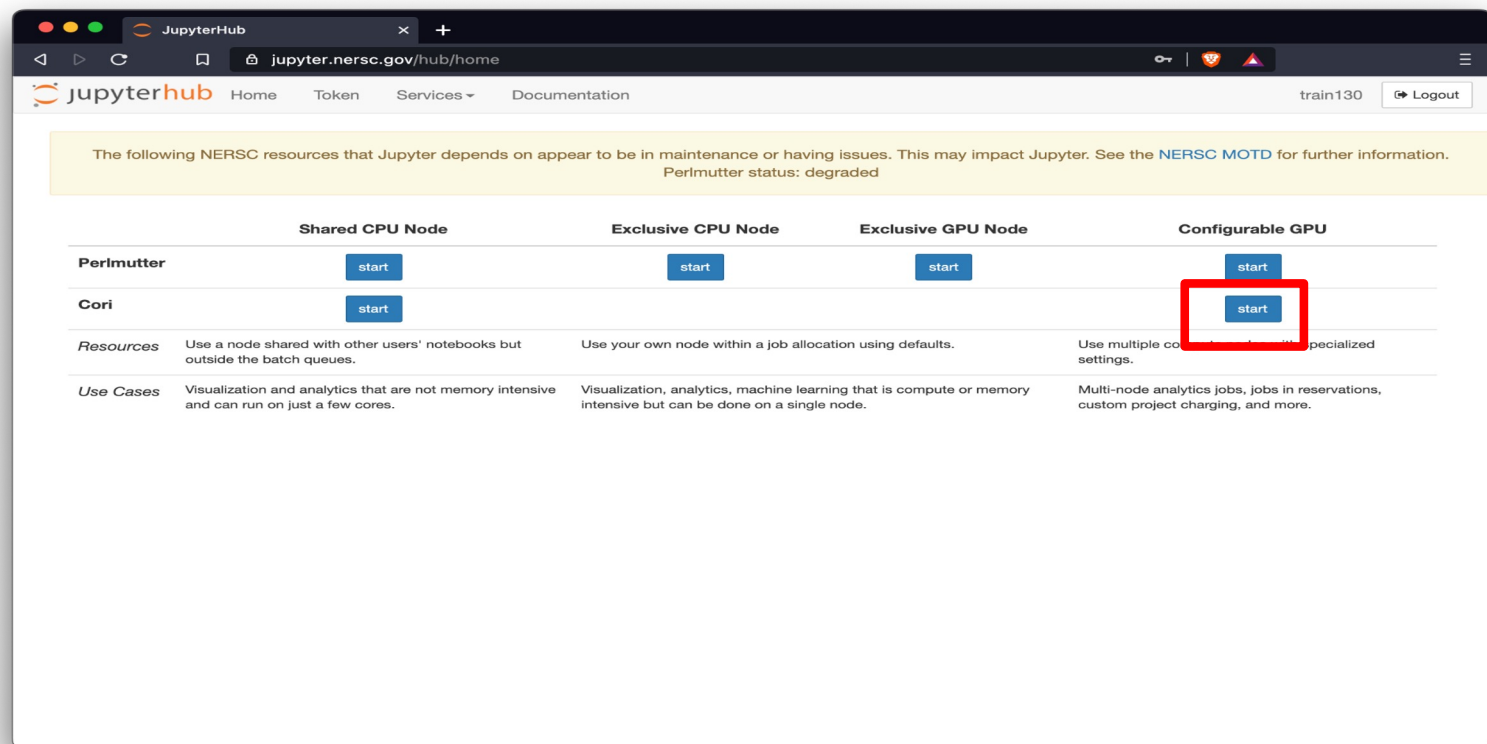
# Setup

- Stop your server by going to: “File” > “Hub Control Panel”
- The selecting “Stop”



# Setup

- Start a fresh exclusive node on Perlmutter

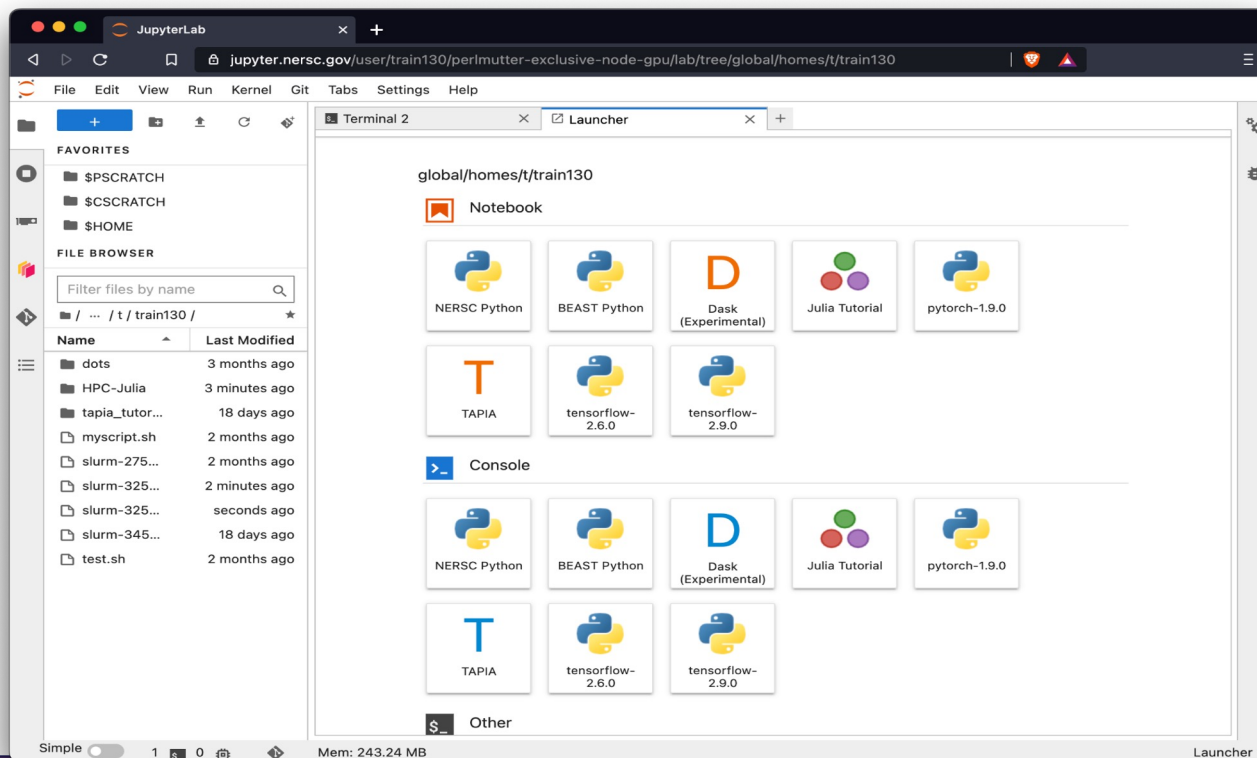


The screenshot shows the JupyterHub interface at `jupyter.nersc.gov/hub/home`. A yellow banner at the top states: "The following NERSC resources that Jupyter depends on appear to be in maintenance or having issues. This may impact Jupyter. See the [NERSC MOTD](#) for further information. Perlmutter status: degraded". Below this, a table lists four node types: Shared CPU Node, Exclusive CPU Node, Exclusive GPU Node, and Configurable GPU. Each type has a "start" button. The "start" button for the Configurable GPU node is highlighted with a red rectangle. Below the table, the "Resources" and "Use Cases" for each node type are described.

|                   | Shared CPU Node                                                                            | Exclusive CPU Node                                                                                               | Exclusive GPU Node    | Configurable GPU                                                                    |
|-------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------|-------------------------------------------------------------------------------------|
| <b>Perlmutter</b> | <a href="#">start</a>                                                                      | <a href="#">start</a>                                                                                            | <a href="#">start</a> | <a href="#">start</a>                                                               |
| <b>Cori</b>       | <a href="#">start</a>                                                                      |                                                                                                                  |                       | <a href="#">start</a>                                                               |
| <b>Resources</b>  | Use a node shared with other users' notebooks but outside the batch queues.                | Use your own node within a job allocation using defaults.                                                        |                       | Use multiple cores and GPUs with specialized settings.                              |
| <b>Use Cases</b>  | Visualization and analytics that are not memory intensive and can run on just a few cores. | Visualization, analytics, machine learning that is compute or memory intensive but can be done on a single node. |                       | Multi-node analytics jobs, jobs in reservations, custom project charging, and more. |

# Setup

- Select the launcher, you should see the Cuda-Q Kernel :)



# Using Reservations in Tutorials

- Go to <https://jupyter.nersc.gov> and select: “Configurable GPU” in the “Perlmutter” row

 Home Token Services Documentation train130 Logout

The following NERSC resources that Jupyter depends on appear to be in maintenance or having issues. This may impact Jupyter. See the [NERSC MOTD](#) for further information.  
Perlmutter status: degraded

|            | Shared CPU Node                                                                            | Exclusive CPU Node                                                                                               | Exclusive GPU Node    | Configurable GPU                                                                    |
|------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------|-------------------------------------------------------------------------------------|
| Perlmutter | <a href="#">start</a>                                                                      | <a href="#">start</a>                                                                                            | <a href="#">start</a> | <a href="#">start</a>                                                               |
| Cori       | <a href="#">start</a>                                                                      |                                                                                                                  |                       | <a href="#">start</a>                                                               |
| Resources  | Use a node shared with other users' notebooks but outside the batch queues.                | Use your own node within a job allocation using defaults.                                                        |                       | Use multiple compute nodes with specialized settings.                               |
| Use Cases  | Visualization and analytics that are not memory intensive and can run on just a few cores. | Visualization, analytics, machine learning that is compute or memory intensive but can be done on a single node. |                       | Multi-node analytics jobs, jobs in reservations, custom project charging, and more. |

# Jupyter Options:

Leave defaults, except:

Account:

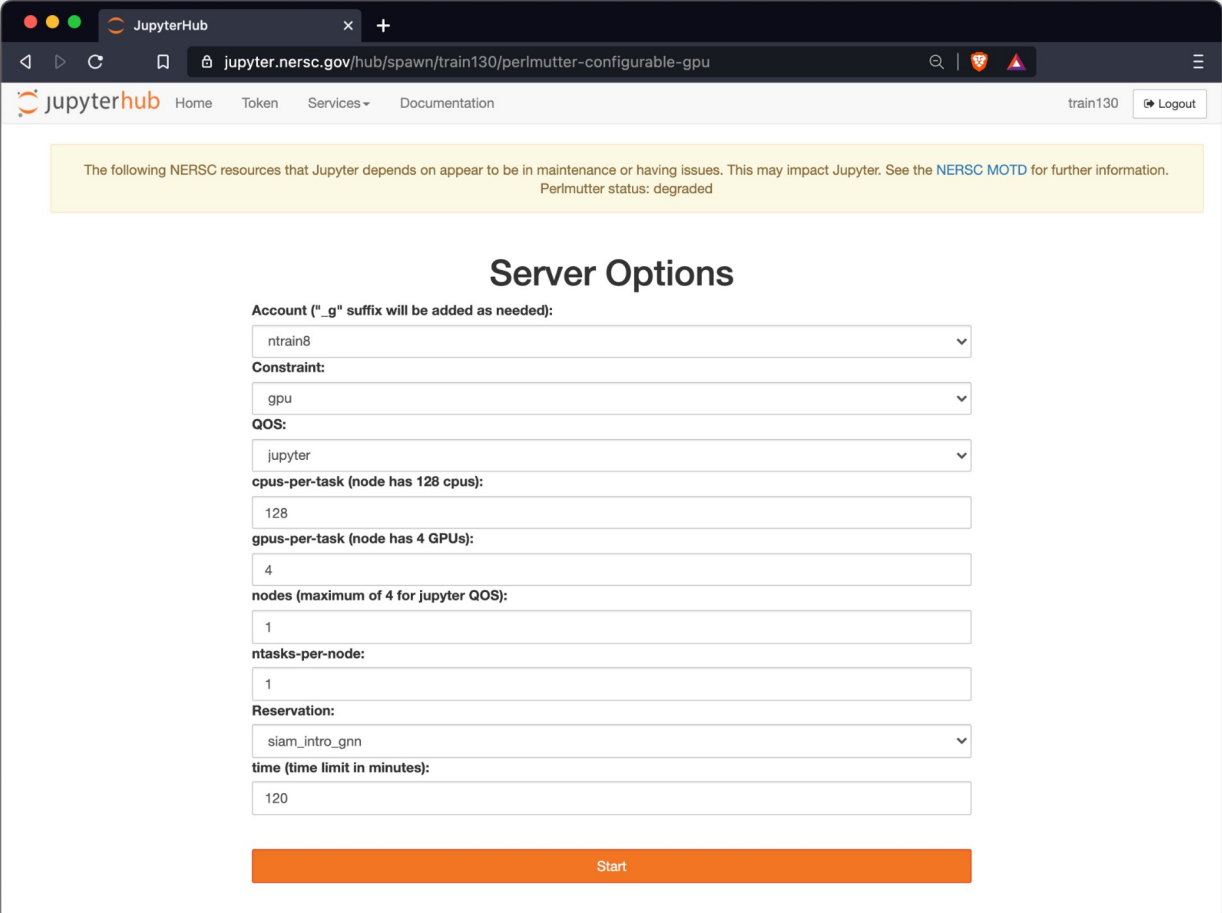
ntrain6

Reservation

TBD

Time

TBD



The screenshot shows the JupyterHub web interface in a browser. The URL is `jupyter.nersc.gov/hub/spawn/train130/perlmutter-configurable-gpu`. The page has a header with the JupyterHub logo and navigation links (Home, Token, Services, Documentation). A yellow banner at the top contains a message about NERSC resources. The main section is titled "Server Options" and contains several form fields for configuring a Jupyter server. The fields are: Account (with a dropdown menu showing "ntrain8"), Constraint (with a dropdown menu showing "gpu"), QOS (with a dropdown menu showing "jupyter"), cpus-per-task (with a text input showing "128"), gpus-per-task (with a text input showing "4"), nodes (with a text input showing "1"), ntasks-per-node (with a text input showing "1"), Reservation (with a dropdown menu showing "siam\_intro\_gnn"), and time (with a text input showing "120"). A large orange "Start" button is at the bottom.

The following NERSC resources that Jupyter depends on appear to be in maintenance or having issues. This may impact Jupyter. See the [NERSC MOTD](#) for further information.  
Perlmutter status: degraded

### Server Options

Account ("\_g" suffix will be added as needed):

ntrain8

Constraint:

gpu

QOS:

jupyter

cpus-per-task (node has 128 cpus):

128

gpus-per-task (node has 4 GPUs):

4

nodes (maximum of 4 for jupyter QOS):

1

ntasks-per-node:

1

Reservation:

siam\_intro\_gnn

time (time limit in minutes):

120

Start

## Note:

- The reservation is active until 5pm today. If you request a node during this time, make sure that the current time + requested time does not exceed the reservation's end time (5pm).
- Training accounts are fully fledged NERSC accounts
- Will remain available until Tuesday – please copy any data and code to your computer before then

## Hour 2

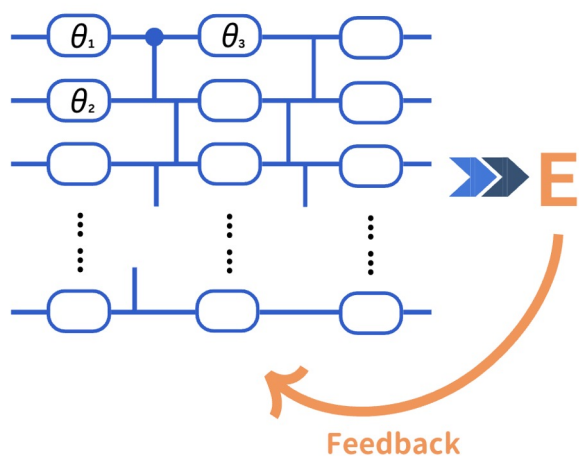
# AI for Scalable Quantum Algorithms : Generative Quantum Eigensolver



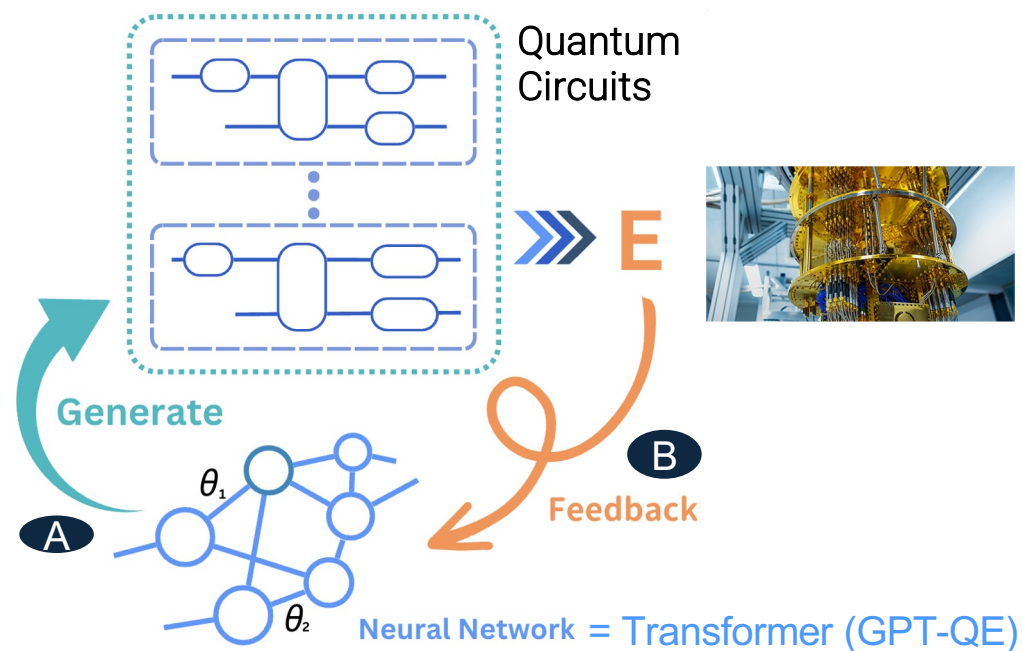
## Generative quantum eigensolver (GQE)

GQE uses the generative model of quantum circuits.

### VQE



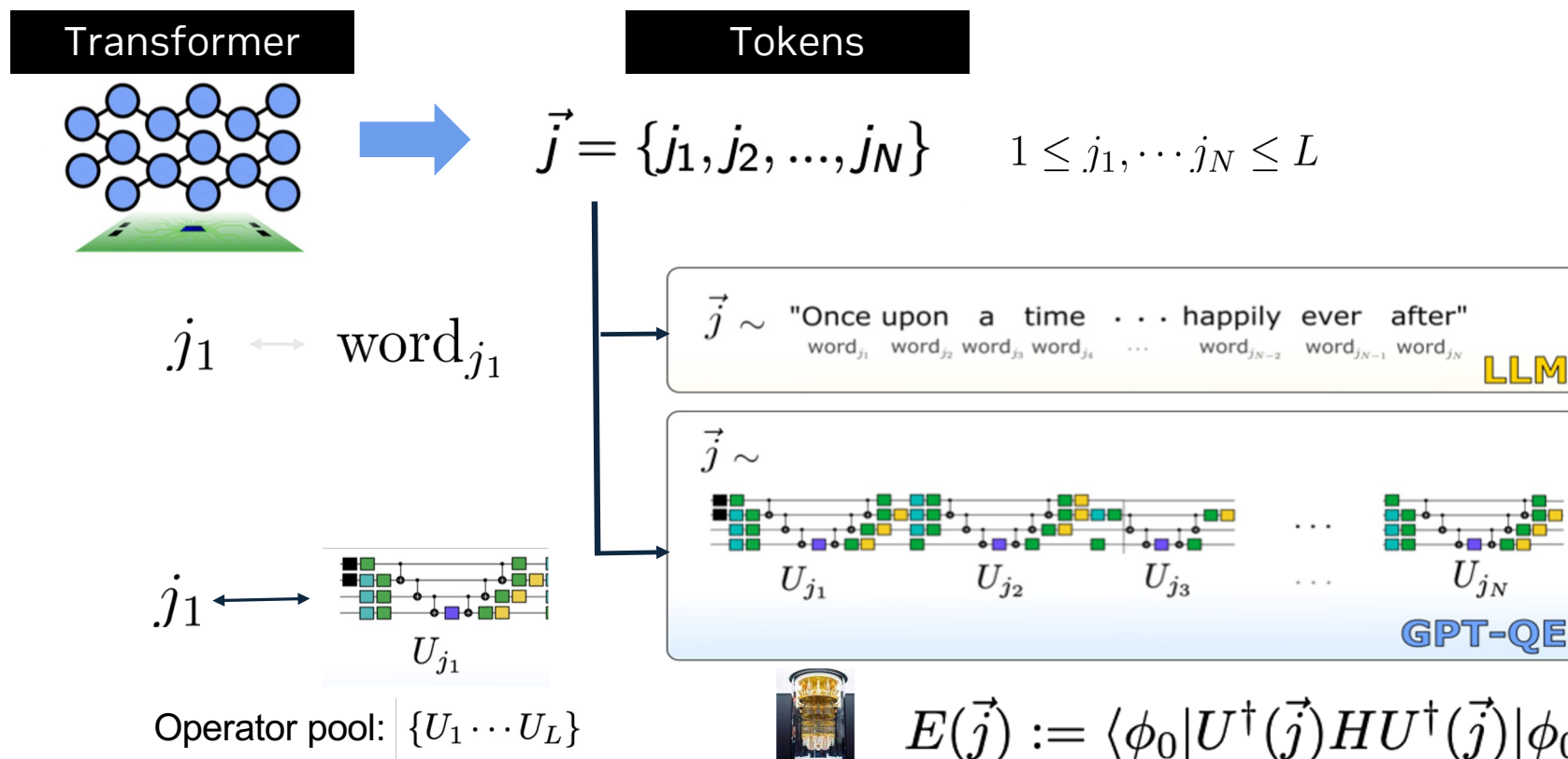
### Generative quantum eigensolver (GQE)



[1] K. Nakaji, et al. "The generative quantum eigensolver (GQE) and its application for ground state search." *arXiv preprint arXiv:2401.09253* (2024).

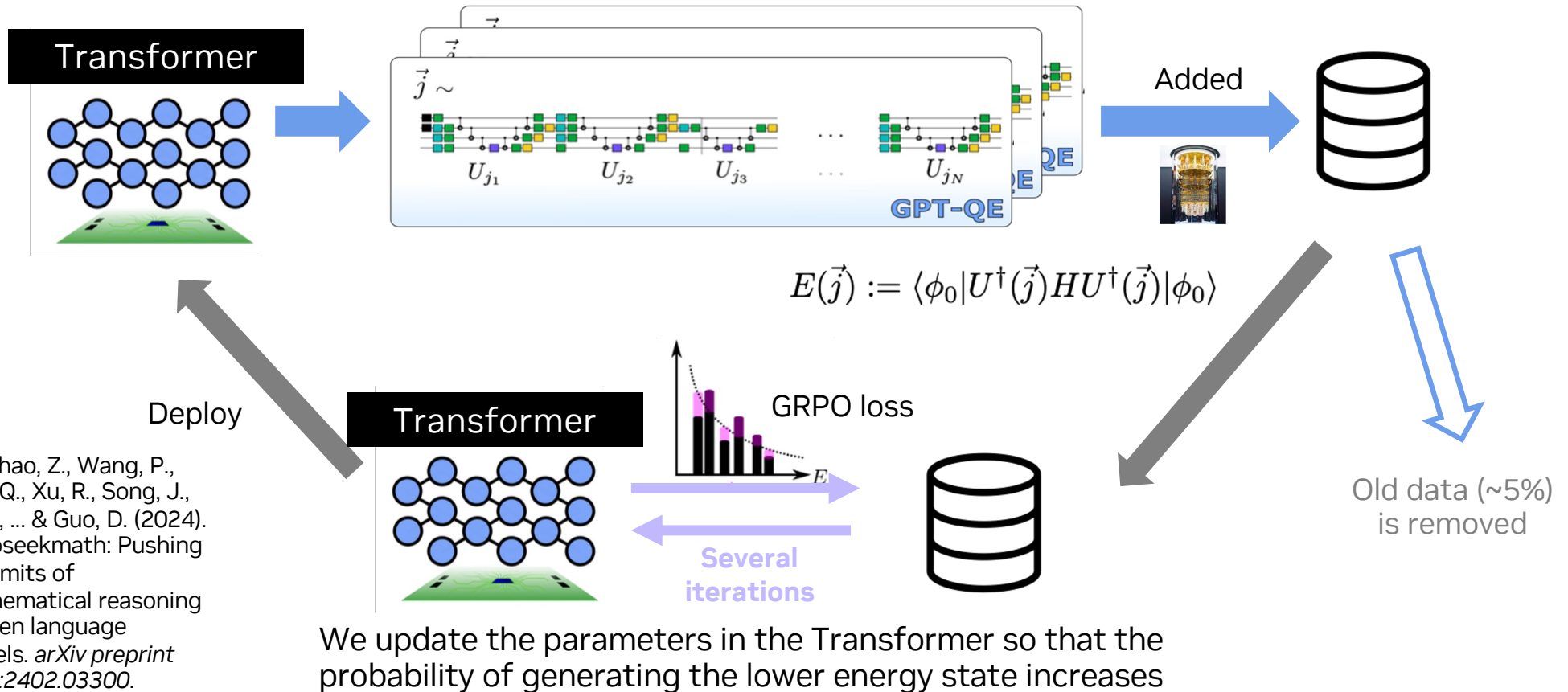
## A How can we generate quantum circuits?

Each generated token is mapped to a quantum circuit component



## B How can we train the model?

We train the model with the dataset generated

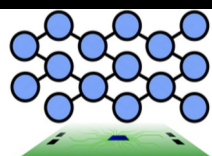


We update the parameters in the Transformer so that the probability of generating the lower energy state increases

# Proof-of-concept: GPT-QE with chemistry Hamiltonians

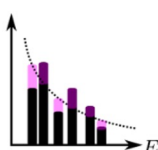
For each atom distance ...

**Transformer**



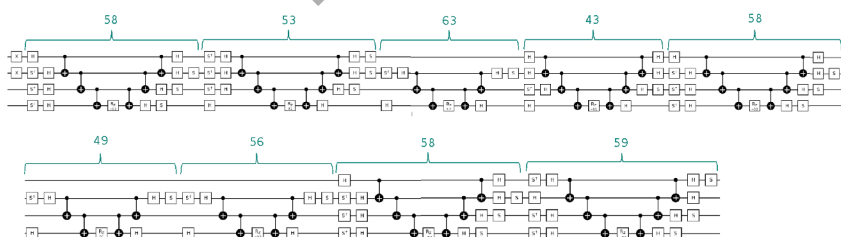
training

**Simulator**



Operator pool:  
Constructed from UCCSD operators

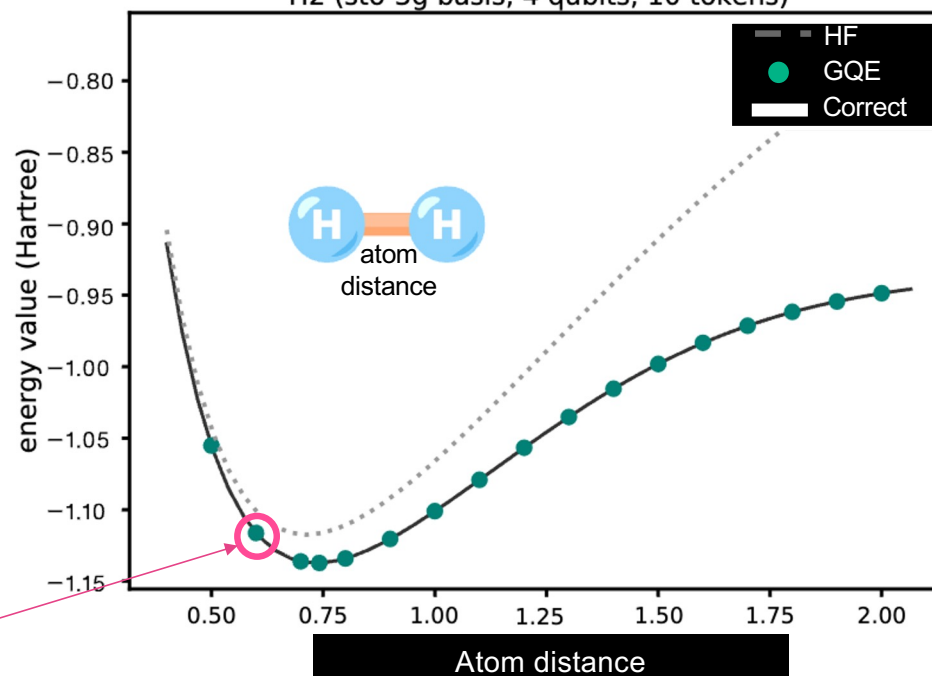
generate



$$E(\vec{j}) := \langle \phi_0 | U^\dagger(\vec{j}) H U(\vec{j}) | \phi_0 \rangle$$

\*Having the form:  $e^{iP_j t_j}$ , where  $P_j$  is a Pauli operator, which appears when Trotterizing UCCSD operators.  $t_j$  is one of the discrete time.

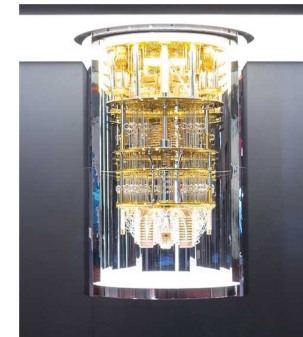
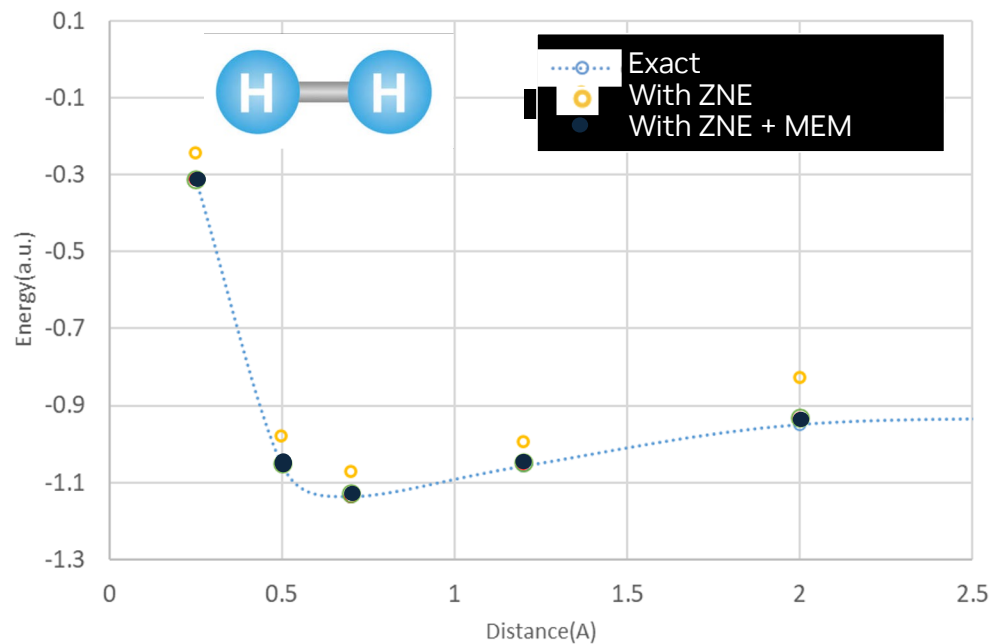
H2 (sto-3g basis, 4 qubits, 10 tokens)



# GPT-QE in actual quantum device

Demonstration in actual quantum device

- GPT-QE in IBM device [4]:



IBM Kawasaki

- ZNE: zero noise extrapolation
- MEM: measurement error mitigation

- Quantinuum announced that they successfully trained GPT-QE on their device [5]

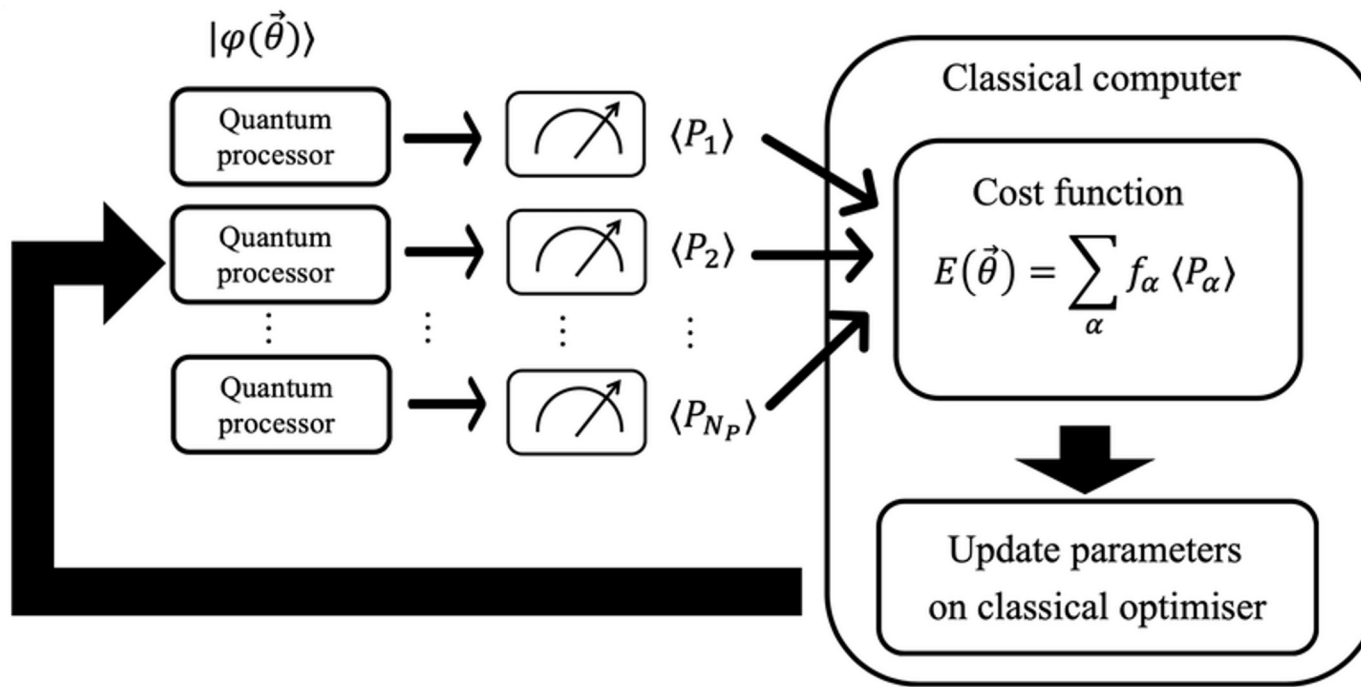
[3] Follow-up work from Qi Gao (Mitsubishi Chemical) and Michihiko Sugawara (Keio Quantum Computing Center)

[4] <https://www.quantinuum.com/blog/genqai-a-new-era-at-the-quantum-ai-frontier>

End of GQE

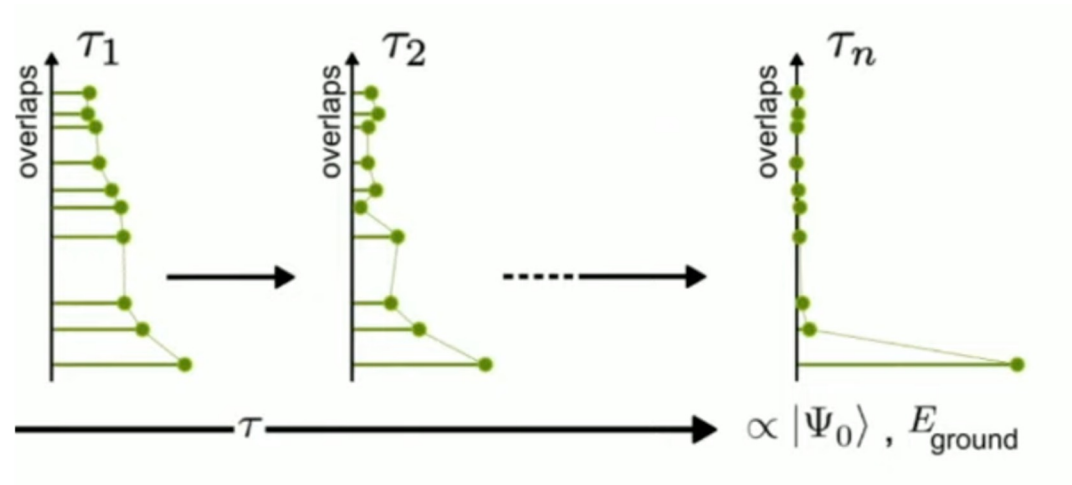
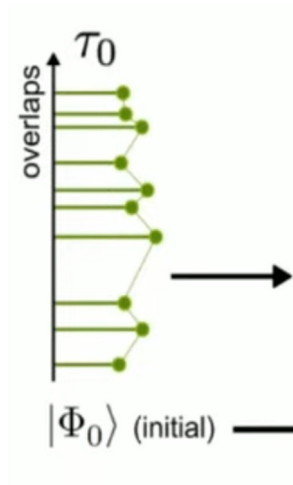
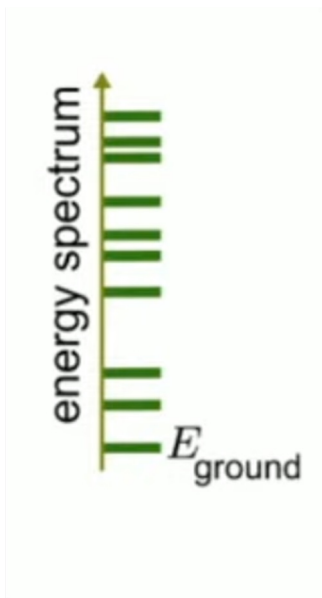


# VQE



## Imaginary Time Evolution

$$|\Psi_0\rangle \propto \lim_{\tau \rightarrow \infty} \exp(-\tau \hat{H}) |\Phi_0\rangle$$



# AFQMC

## Auxiliary Field Quantum Monte Carlo

- Discretize time
- Trotterize
- Sum the squares
- Propagate a timestep
- Add the auxiliary fields
- Thouless theorem
- Monte carlo sample the high dimensional integral

$$|\Psi_0\rangle \propto \lim_{\tau \rightarrow \infty} e^{-\tau \hat{H}} |\Phi_0\rangle \simeq \lim_{N \rightarrow \infty} \left( e^{-\Delta\tau \hat{H}} \right)^N |\Phi_0\rangle$$

$$\hat{H} = \hat{H}_1 + \hat{H}_2 \quad e^{-\Delta\tau \hat{H}} = e^{-\Delta\tau \hat{H}_1} e^{-\Delta\tau \hat{H}_2} + \mathcal{O}(\Delta\tau^2)$$

$$\hat{H}_2 = \frac{1}{2} \sum_{\alpha} \hat{v}_{\alpha}^2$$

$$e^{-\Delta\tau \hat{H}_1} e^{-\Delta\tau \hat{H}_2} |\Phi_0\rangle = e^{-\Delta\tau \hat{H}_1} e^{-\frac{\Delta\tau}{2} \sum_{\alpha} \hat{v}_{\alpha}^2} |\Phi_0\rangle$$

$$e^{\frac{a^2}{2}} \propto \int_{-\infty}^{\infty} dx e^{-\frac{x^2}{2}} e^{-ax} \quad e^{-\Delta\tau \hat{H}_2} \propto \int d^M \mathbf{x} e^{-\frac{1}{2} \mathbf{x}^{\dagger} \mathbf{x}} e^{-i\sqrt{\Delta\tau} \sum_{\alpha} \hat{v}_{\alpha} x_{\alpha}}$$

$$e^{-\Delta\tau \hat{H}_1} e^{-i\sqrt{\Delta\tau} \sum_{\alpha} \hat{v}_{\alpha} x_{\alpha}} |\Phi_0\rangle \propto |\tilde{\Phi}_0\rangle$$

$$|\Psi_0\rangle \propto \left( e^{-\Delta\tau \hat{H}_1} \int d^M \mathbf{x} e^{-\frac{1}{2} \mathbf{x}^{\dagger} \mathbf{x}} e^{-i\sqrt{\Delta\tau} \hat{V}(\mathbf{x})} \right)^{\infty} |\Phi_0\rangle$$

# VQE - AFQMC Results for H10 molecule

20 qubits on 16 nodes

R=1.0 , FCI Corr Energy = 167.78 mHartree

| Ansatz | # of Params | # of Iterations | VQE Corr Energy (mHA) | # of Determinants | AFQMC Corr Energy (mHa) |
|--------|-------------|-----------------|-----------------------|-------------------|-------------------------|
| UCCSD  | 875         | 5               | 165.02                | 58559             | 167.49 $\pm$ 0.03       |
| UpCCD  | 25          | 8               | 34.39                 | 252               | 163.28 $\pm$ 0.60       |

R=2.0 , FCI Corr Energy = 791.17 mHartree

[Github link](#)

| Ansatz | # of Params | # of Iterations | VQE Corr Energy (mHA) | # of Determinants | AFQMC Corr Energy (mHa) |
|--------|-------------|-----------------|-----------------------|-------------------|-------------------------|
| UCCSD  | 875         | 18              | 767.98                | 63502             | 787.46 $\pm$ 0.35       |
| UpCCD  | 25          | 9               | 74.16                 | 252               | 628.27 $\pm$ 4.65       |

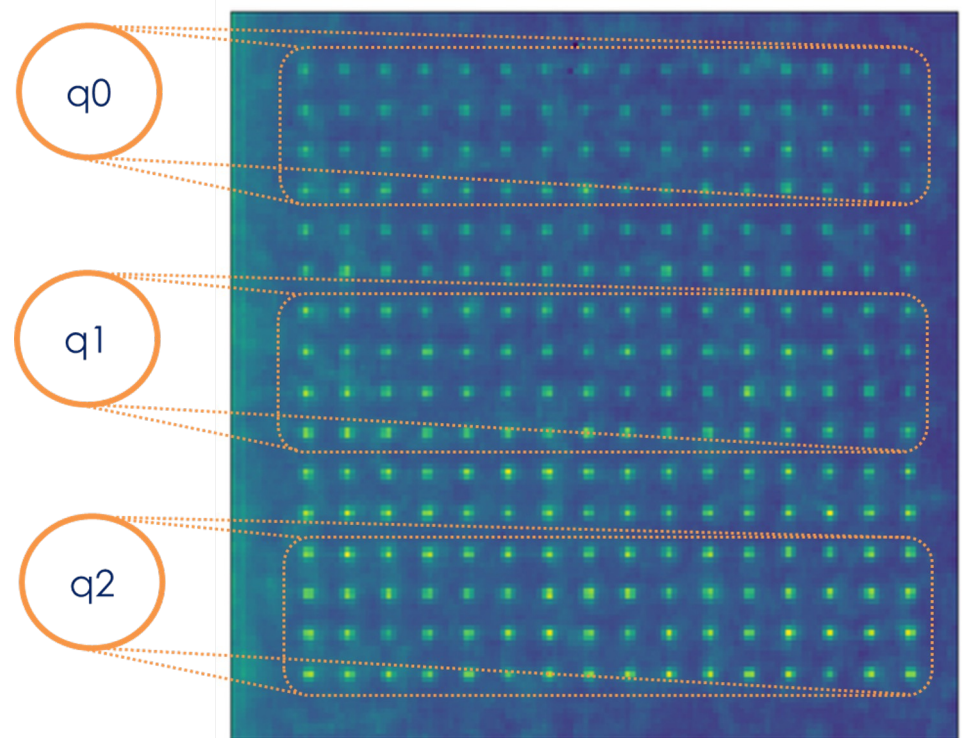
**Hour 3**

## **Logical Qubit Experiments**



## Logical qubits in quantum computation

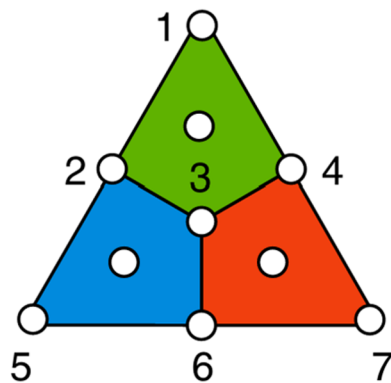
- A logical qubit is collection of physical qubits protected by a QECC (quantum error correcting code)
- “Many noisy qubits → a noise resilient, reliable collective qubit”
- Logical qubits enable longer and more accurate computation by decreasing the logical error rate



## Logical qubits: entering E-FTQC

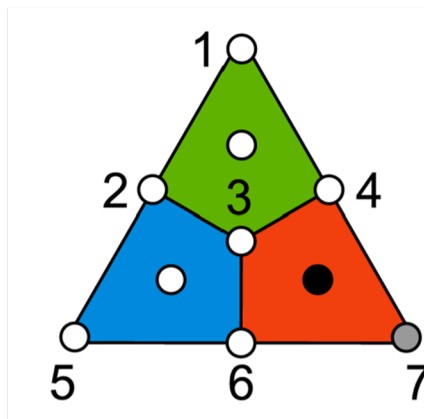
- Going from NISQ (noisy intermediate-scale quantum) systems to E-FTQC (early - fault tolerant quantum computation) systems
- Doing so requires a QEC workflow: encoding, syndrome extraction, decoding

### • Encoding



[PHYS.REV.X11,041058(2021)]

### • Check syndrome

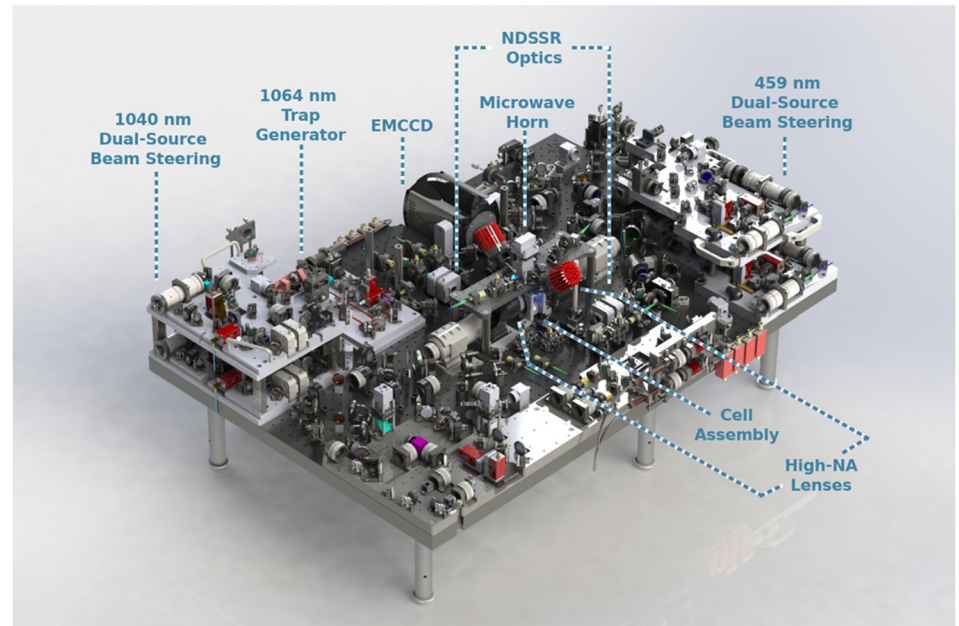
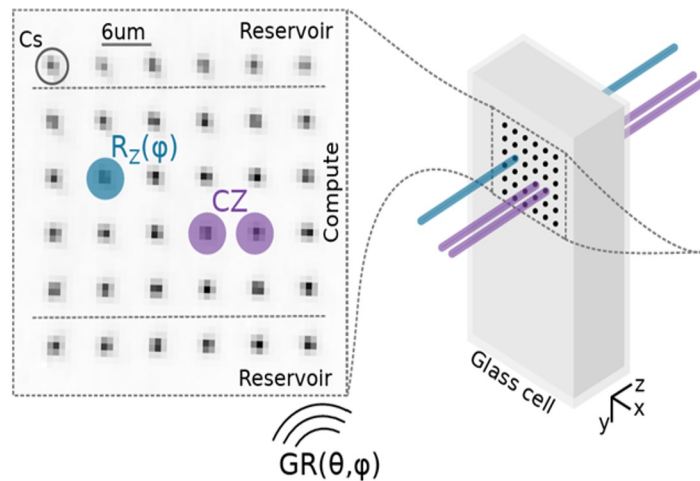


### • Decoding

| Flag 0 | Flag 1 | Flag 2 |
|--------|--------|--------|
| 0      | 0      | 0      |
| 0      | 0      | 1      |
| 0      | 1      | 0      |
| 0      | 1      | 1      |
| 1      | 0      | 0      |
| 1      | 0      | 1      |
| 1      | 1      | 0      |
| 1      | 1      | 1      |

## Logical qubits demonstrated on Infleqtion's Sqale QPU

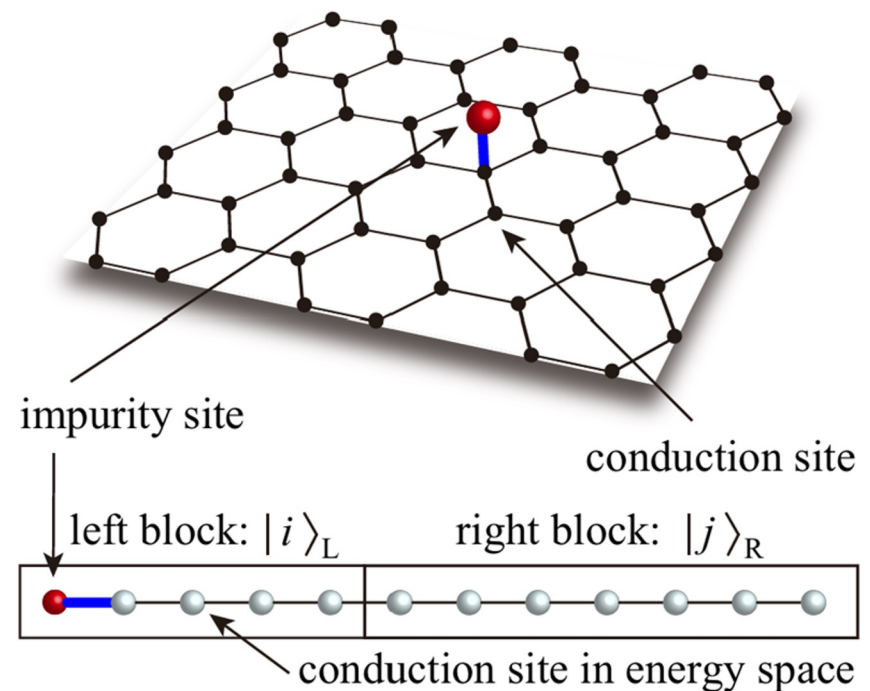
- Image array of atoms and laser beams implementing quantum gates:



3D Model of Infleqtion's gate-based neutral atom quantum processing unit (QPU): Sqale

## Applications with Logical Qubits: Materials Science

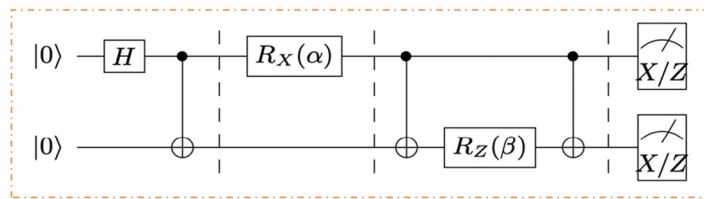
- Simulate physics of Anderson Impurity Model (AIM) on a quantum computer
- Study strongly correlated interactions of an 'impurity' introduced to a 'bath'
  - Improved simulations of correlated materials
  - Design of novel material properties



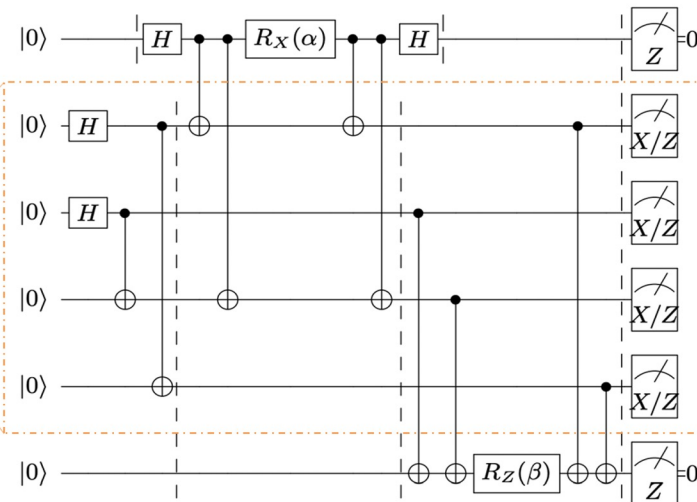
[PhysRevB.93.205124]

## Materials Science with VQE on $[[4,2,2]]$ Logical Qubits

- Using Hamiltonian Variational Ansatz (HVA) to variationally prepare the Anderson Impurity Model (AIM) ground state
  - Underlying Hamiltonian used for VQE has broader applications: excluding the Pauli-X terms yields an Ising-like Hamiltonian which can be used to solve classical optimization problems
- Variational optimization done with GPU support -> hybrid quantum-classical interplay

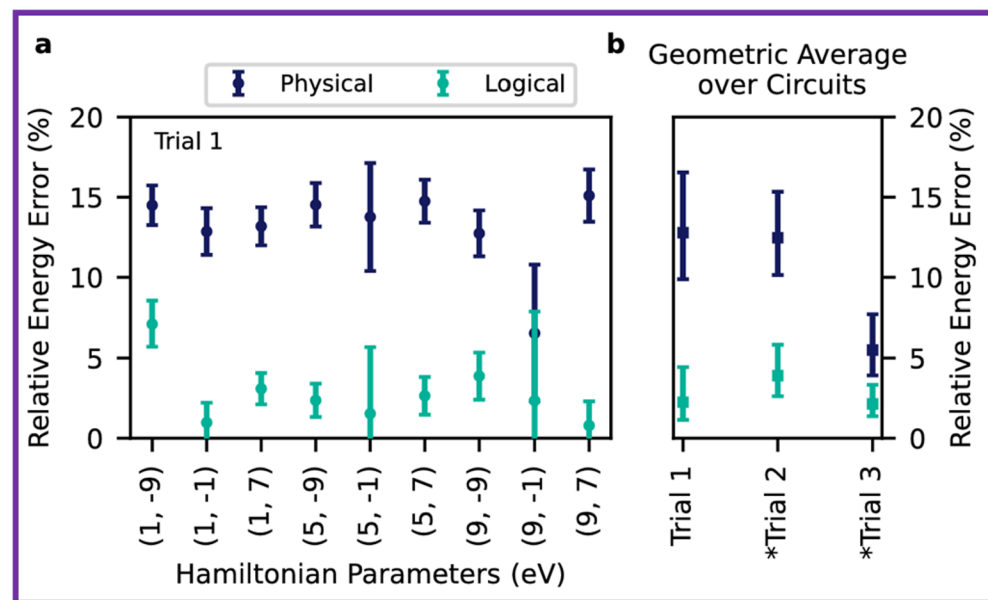
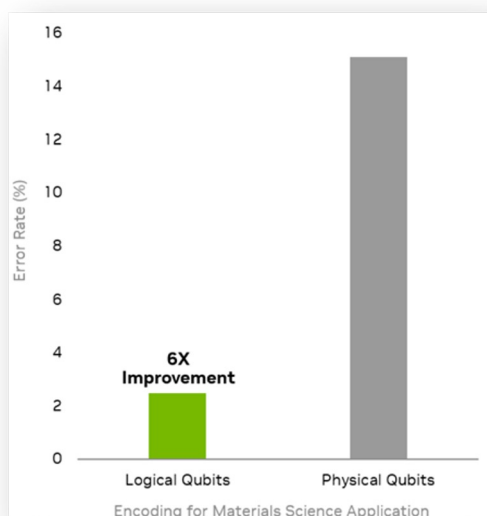


$$\mathcal{U}(\alpha, \beta) = e^{-i\beta Z_0 Z_2 / 2} e^{-i\alpha X_0 / 2}$$



## Logical qubit AIM experimental demonstration

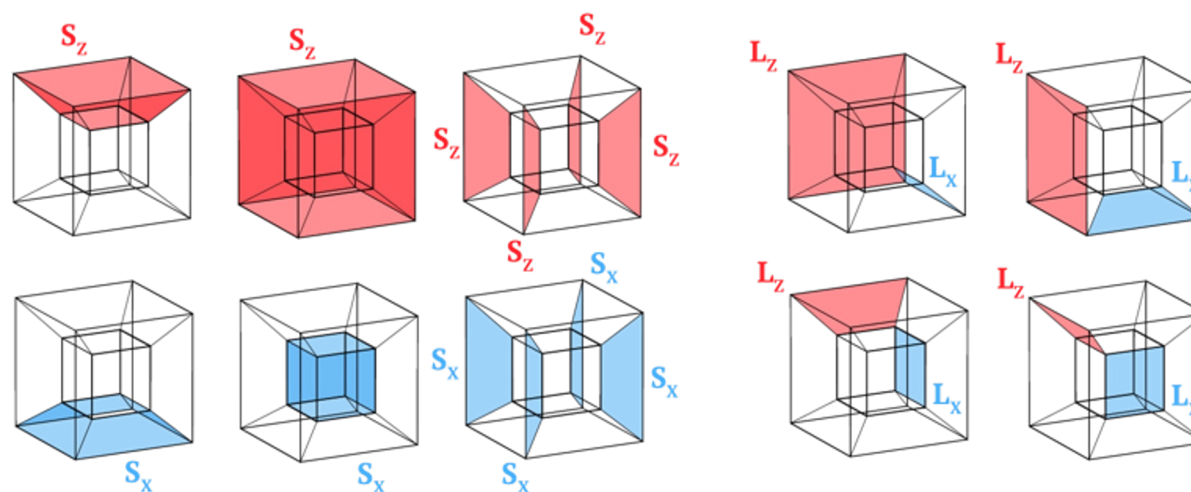
- Results from running logical circuits on Sqale on the right
- Different Hamiltonian parameters correspond to different circuit sets



Experimental results from <https://arxiv.org/abs/2412.07670>

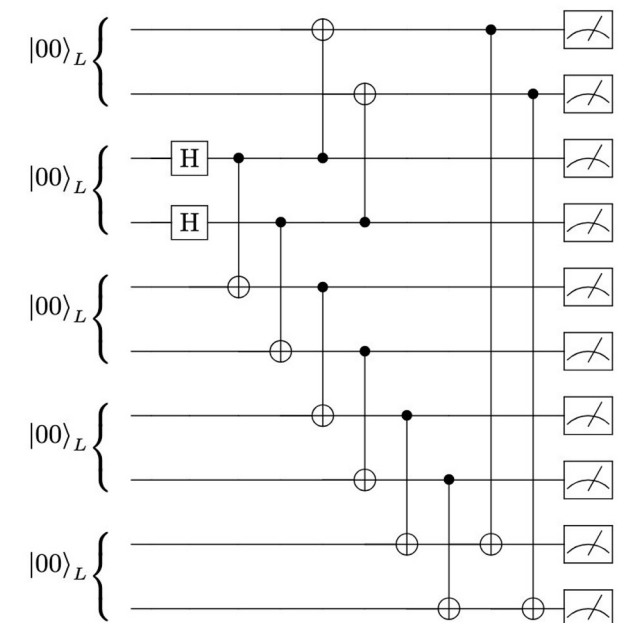
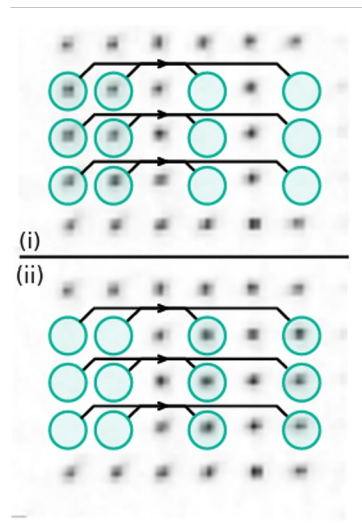
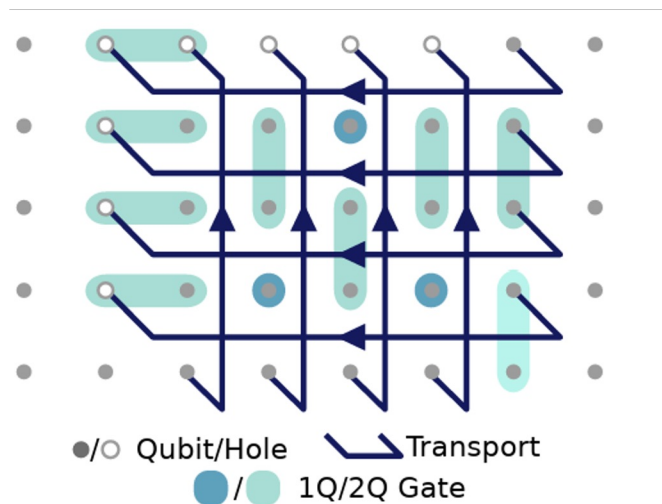
## More logical qubits with high-rate QECCs: many-hypercube code

- Infleqtion has implemented the  $[[16, 4, 4]]$  concatenated code, based on Goto's original many-hypercube code (Science Advances 10.36 (2024): eadp6388.)
- Encodes 4 logical qubits at distance 4 with only 16 physical qubits (high-rate)
- Foundation for efficient QEC schemes as a high-rate, high-threshold quantum error correcting code



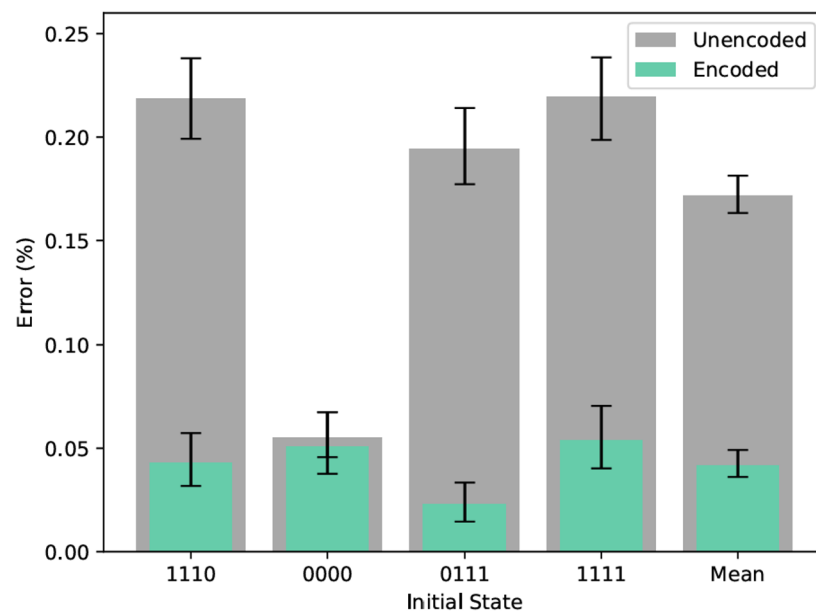
## Many hypercube code on Sqale with atom motion

- Fault tolerant preparation of the  $[[16, 4, 4]]$  code requires *long range connectivity* – made possible on Sqale via atom (qubit) movement



## Many hypercube code experimentally realized on Sqale

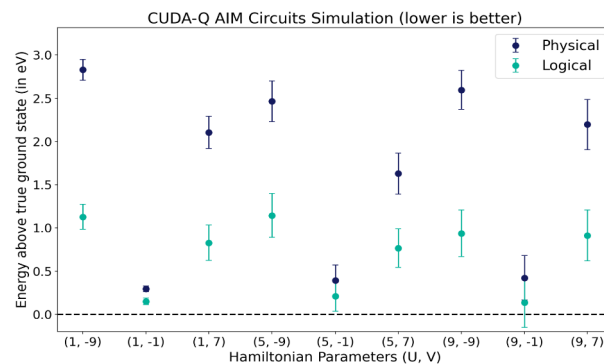
- Prepared the logical states in the  $[[16,4,4]]$  code
- Applied transversal gates for specific logical states
- Measurements post-processed with loss detection and syndrome decoding



- Better logical-than-physical performance validated for multiple state preparation experiments. More details in <https://arxiv.org/abs/2509.13247>

## Logical experiments tutorial outline

1) Reproduce logical materials science VQE experimental results in CUDA-Q

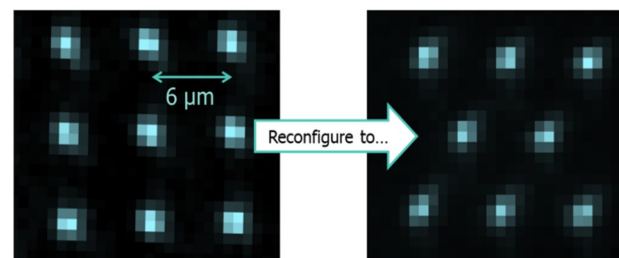
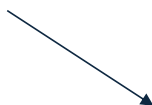


1) Reproduce many-hypercube code results via GPU-accelerated decoder using experimental data from Sqale



```
1 data_bits = np.asarray(list(map(int, bitstring)), dtype=np.uint8)
2 syndrome = parity_check_mat.dot(data_bits) % 2
3
4 results = nv_gpu_decoder.decode(syndrome)
5
6 dec_result = np.asarray(results.result, dtype=np.uint8).ravel()
7 if dec_result.sum() == 1:
8     corrected_bits = data_bits ^ dec_result
9     return corrected_bits
```

1) Experimentally demo atom rearrangement on Infleqion's Sqale QPU via submission to the atom picture API



**Hour 3**

**Logical Qubit Experiments End**



**Brev Coupon Code**

**sc-25-quantum**